2章
移行を計画する

モノリスの分解における細かい技術的な側面に真っすぐ飛び込むのは簡単だ。そして、それが本書の残りの焦点でもある。しかし、その前に、あまり技術的ではないいくつかの課題を探る必要がある。移行をどこから始めるべきか？　変化をどのように管理すべきか？　どのようにして他の人を旅に連れて行くのか？　そして、早い段階で問われるべき重要な質問はこうだ。そもそもマイクロサービスを用いるべきなのだろうか？

2.1　目的を理解する

マイクロサービスは目的ではない。マイクロサービスを手にすることが「勝利」ではないのだ。マイクロサービスアーキテクチャを採用することは、合理的な意思決定に基づいた意識的な判断であるべきだ。マイクロサービスアーキテクチャへの移行は、既存のシステムアーキテクチャのままでは達成できないことを実現するために考えるべきだ。

何を達成しようとしているかを把握せずに、どの選択肢を取るべきかの意思決定プロセスを、どうやって方向付けるのだろうか？　どこに時間を集中するのか、どのように優先順位をつけるのかは、マイクロサービス化で達成しようとしていることによって大きく変わってくるはずだ。

目的を理解していないと、選択のプレッシャーから情報の分析に時間をかけ過ぎて、判断を下せない状態に陥ってしまったり、あるいは、「マイクロサービスがNetflixにとって良いものなら、自分たちにとっても良いもののはずだ！」と思い込んで、カーゴカルト[†1]的なメンタリティに陥ってしまう危険性もある。

[†1] 訳注：積荷信仰ともいう。本質を理解せず表層のみを模倣する人々や行為を指す。

よくある失敗

数年前、あるカンファレンスでマイクロサービスのワークショップを開催していた時のことだ。どの研修でもそうだが、私は参加者がなぜそこにいるのか、そして何を持ち帰りたいかということを知るのが好きだ。その研修では何名かが同じ会社から参加していたため、私はその理由を知りたいと思った。そのうちの1人に「あなた方がワークショップに参加した理由は何ですか？　なぜマイクロサービスを使うことに興味があるのでしょう？」と尋ねてみた。すると、彼はこう答えた。「わかりません。上司が参加しろと言ったので」 興味をそそられた私はさらに訊いてみた。「上司がここにあなたを送り込んだ理由に何か心当たりはありませんか？」 すると、こういう答えが返ってきた。「そうですね、彼に直接訊いてみたらどうでしょう。彼は私たちの後ろに座っていますよ」 私は質問する相手を上司に変えて、同じ質問をした。「それで、なぜマイクロサービスを使用するのでしょう？」 上司はこう答えた。「うちのCTOがマイクロサービスをやると言ったので、それが何なのかを知るべきだと思ってここに来ました」

この実話は、ある意味では面白いのだが、残念ながらあまりにもよくある話だ。なぜそれを採用し、何を実現しようとしているのかを本当に理解していないまま、マイクロサービスアーキテクチャの採用を決定している多くのチームと、私はよく遭遇する。

マイクロサービスを使う理由を明確に持っていないことの問題点はさまざまだ。マイクロサービス化は、そのために投じる人員や費用という直接的な意味でも、フィーチャーの追加よりも移行作業を優先するという意味でも、多額の投資が必要になる。これは、移行のメリットを実感するまでに時間がかかるという事実によって、さらに状況を複雑にする。時として、それは移行に1年以上かけているにもかかわらず、なぜそれを始めたのかを思い出せなくなってしまうという状況へとつながってしまう。これはサンクコストの錯誤ではない。なぜなら、なぜ仕事をしているのかを、彼らは文字通り理解していないからだ。

一方で、私はマイクロサービスアーキテクチャに移行した場合の費用対効果（ROI）を教えてほしいという依頼を受けることもある。ある人たちは、なぜこのアプローチを検討するかにさえ、裏付けとなる確固たる事実と数字を求めている。この種のことがらについての詳細な研究はほとんどないという事実はさておき、たとえ存在する場合にさえ、現実にはそうした研究結果を活かせることは滅多にない。それぞれが置かれている状況はさまざまだからだ。

では、私たちにできるのは憶測だけなのだろうか？　そうじゃない。私たちの開発、技術、アーキテクチャの選択の有効性について、より良い研究ができると確信しているし、そうすべきだ。そのための研究も既にいくつか行われている。“The State of DevOps Report（DevOpsの現状に関するレポート）”（http://bit.ly/2ojVq5o）などがそうだ。しかし、そうした研究は付随的にアーキテクチャにも目を向けているにすぎない。このような厳密な研究の代わりに、私たちは少なくとも、意思決定の際にもっとクリティカルシンキングを行うよう努力すべきであり、同時に、より実験的な考え方を受け入れなくてはならない。

私たちは何を達成したいのかを明確に理解する必要がある。求めているものが何であるかを適切に評価しなければ、ROIは計算できない。達成したい成果に焦点を当てる必要があるが、独断的に一つのアプローチに固執する必要はない。たとえそれが多くの仕事を捨てることや、古き良き退屈なやり方に逆戻りすることを意味するとしても、必要なものを手に入れるための最善の方法について、明確かつ賢明に考える必要がある。

2.1.1　3つの重要な質問

マイクロサービスアーキテクチャ化を検討すべきかを組織が見極める支援を行う際、私は次の質問をすることにしている。

達成したいことは何か？
事業が達成しようとしていることに合った一連の成果が目的でなければならない。これは、システムのエンドユーザーへのメリットを説明する方法で明確にできる。

マイクロサービスの他に代替案はなかったか？
後で説明するように、マイクロサービスがもたらす利点は、別の方法でも実現できることが多い。そうした方法は検討しただろうか？　そうでない場合、それはどうしてだろうか？　たいていは、ずっと簡単で退屈な方法によって、必要なものを手にすることが可能だ。

どうすれば移行がうまくいってるか分かるだろうか？

移行に着手することを決めた場合、どうすれば正しい方向に進んでいるかが分かるだろうか？　このトピックについては、章の最後で説明する。

これまでの経験上、マイクロサービス化の検討をさらに進めるかどうかを企業にもう一度考えさせるには、これらの質問があれば十分だ。

2.2　マイクロサービスを選択する理由

私にはあなたの会社の目標は定義できない。会社が掲げている野望と直面している課題は、あなたの方がよくわかっているはずだ。ここでは、マイクロサービスが世界中の企業で採用されている理由としてよく挙げられるものを説明する。そして、誠実さの精神に従って、異なるアプローチによって同じ結果を達成できる可能性のある他の方法についても説明していく。

2.2.1　チームの自律性を高める

> *どんな業界で働いていたとしても、人が全てだ。彼らがうまくやっているところを見つけ、彼らが真の可能性に達するよう自信、動機、自由、願望を提供しなさい。*
>
> *John Timpson*

自律的なチームを作ることの利点は、多くの組織によって示されている。そうした組織は、グループの規模を小さく保ち、官僚主義をあまり持ち込まないようにすることで、緊密な絆を築いて効果的に仕事を行えるようにしている。そうすることで、他の組織よりも効果的に成長し、規模の拡大を実現している。たとえばGore社は、全員が確実に知り合いとなるよう、いずれのビジネスユニットも150人を超えないようにして、大きな成功を収めている。そうした小規模なビジネスユニットが機能するには、自律的なユニットとして機能するための権限と責任が与えられなければならない。

イギリスの小売業者として大成功を収めているTimpsons社は、従業員に権限を与え、中央機能の必要性を減らし、不満を抱えた顧客への返金額の決定などを地域の店舗が自ら下せるようにすることで、大規模な拡大を実現した。現在、会社の会長であるJohn Timpsonは、社内ルールを廃止し、次の3つだけに置き換えたことで有名な人物だ。

- 身だしなみを整えること。

- レジにお金を入れておくこと。
- お客さまに最善のサービスを提供するためなら何でもすること。

自律性は小さな規模でも機能する。私が一緒に仕事をしている現代的な組織の多くは、組織内でより自律的なチームを作ろうとしていて、Amazonの2枚のピザ理論やSpotifyモデル[†2]といった他の組織のモデルをコピーしようとしていることが多い。

[†2] 有名な話だが、Spotifyも今はもうそのモデルは使っていない。

それが適切であれば、チームの自律性は人々に力を与え、チームメンバーがステップアップして成長し、仕事をより早く終わらせる助けになる。チームがマイクロサービスを所有し、それらのサービスを完全にコントロールできるようになれば、より大きな組織の中でチームが持つ自律性は高まる。

他にどんな方法があるか

自律性、つまり責任の分配は、さまざまな方法で実現可能だ。より多くの責任をチームに持たせる方法を考えるにあたって、アーキテクチャの変更は必須ではない。それはつまり、どのような責任をチームに持たせられるかを特定するプロセスであり、さまざまな方法で実行可能だ。コードベースの一部の所有権を異なるチームに与えることが一つの答えになる可能性もある（モジュラーモノリスを使用するメリットはここにもある）。コードベースの一部の決定を行う権限を誰が持つべきかを、能力的な理由に基づいて決定することもできる（たとえば、Ryanはディスプレイ広告を最もよく知っているので、その責任者となる。Janeはクエリのパフォーマンスのチューニングを最もよく知っているので、この分野のことは何でも彼女に任せるといった具合だ）。

自律性の向上は、単純に、他の誰かがあなたのためにする作業を待つ必要がなくなることにもつながる。たとえば、マシンや環境のプロビジョニングにセルフサービスのアプローチを採用することで、中央の運用チームが日々の活動のためにチケットを処理する必要性を回避できる。

2.2.2　市場投入までの時間を減らす

個別のマイクロサービスに変更を加え、リリースの調整を待たずにそれらの変更をデプロイできるようにすることで、機能をより迅速に顧客にリリースできるようになる。より多くの人々を問題の解決に参加させられるようになることも、市場投入までの時間を減らす要因となる。これについては後述する。

他にどんな方法があるか

さて、どこから始めればいいだろうか？　ソフトウェアをより迅速に出荷する方法を考えるにあたっては、とても多くの変数が存在する。最大の障壁は、実はあなたが思っているものではないかもしれない。こういうときには、私はいつも、本番環境までの経路をモデリングすることをまず提案している。最大の障壁を明らかにする手助けになるかもしれないからだ。

何年も前、ソフトウェアのデリバリーを迅速化するため、ある大規模な投資銀行のプロジェクトに呼ばれたときのことを覚えている。「開発者は本番環境まで持っていくのに時間がかかりすぎている！」 私たちはそう聞かされていた。同僚のKief Morrisは、時間をかけてソフトウェアのデリバリーに関わる全てのステップを書き出し、プロダクトオーナーがアイデアを思いついた瞬間から、そのアイデアが実際に製品化されるまでのプロセスを見ていった。

彼は、開発者が作業を開始してから本番環境にデプロイされるまで、平均して約6週間かかることをすぐに明らかにした。手動のプロセスが関与していたので、適切な自動化を行うことで、このプロセスを2、3週間短縮できるだろうと私たちは感じていた。しかし、Kiefは本番環境への道のりにもっと大きな問題を発見した。それは、プロダクトオーナーがアイデアを思いついてから開発者が作業に着手できる状態になるまでに、40週間以上かかっていることが多いという点だった。そこで、私たちはプロセスのこの部分の改善に注力することにした。これによって、新機能の市場投入までの時間を改善することに関して、大きな支援を達成できた。

ソフトウェアのデリバリーに関わる全てのステップを考えよう。経過時間と実処理時間の両方の観点でステップごとにどれくらいの時間がかかっているのかを見て、過程における問題点を見つけだそう。その結果、マイクロサービスが解決策の一部になり得るかもしれないが、他にも多くのことを並行して試せることに気が付くはずだ。

2.2.3　負荷への費用対効果が高いスケーリング

処理を個々のマイクロサービスに分割することで、これらのプロセスを独立してスケールアップできる。これは、うまくいけば、現在負荷を処理する能力を制限している部分だけをスケールアップすればいいという、費用対効果の高いスケーリングが可能であることを意味している。負荷の低いマイクロサービスをスケールダウンでき、必要のないときには停止もしておける。マイクロサービスアーキテクチャを採用することで、運用コストをよりコントロールできるようになる。これがSaaS製品を構築する多くの企業がマイクロサービスアーキテクチャを採用している理由の1つだ。

他にどんな方法があるか

これには検討すべき多くの代替案がある。その多くはマイクロサービス指向のアプローチを採用する前に実験しやすいものだ。手始めに大きな箱を手に入れることもできるだろう。たとえば、パブリッククラウドやその他の仮想化プラットフォームを利用している場合は、プロセスを実行するためのより大きなマシンをプロビジョニングできる。この「垂直」スケーリングには明らかに限界があるものの、短期的な改善のためには、この方法をすぐさまに否定すべきではない。

既存のモノリスの伝統的な水平スケーリング（基本的には複数のコピーを実行する方法）が、とても効果的だと証明される可能性もある。ロードバランサーやキューのような負荷分散メカニズムの後ろでモノリスの複数のコピーを実行することで、より多くの負荷を簡単に処理できるようになるかもしれない。ただし、ボトルネックがデータベースにある場合には、この方法は役に立たないこともあるだろう。というのも、データベースはそれが基盤とする技術によっては、そのようなスケーリングの仕組みをサポートしていないかもしれないからだ。それでも、水平スケーリングは簡単に試せるため、マイクロサービスを検討する前には試してみるべき方法だ。時間をかけずにその適合性を評価でき、本格的なマイクロサービスアーキテクチャよりも欠点がはるかに少ないからだ。

また、使用している技術を、負荷をより適切に処理できる代替技術に置き換えることもできる。しかし、これは一般的には簡単な作業ではない。既存のプログラムを新しいタイプのデータベースやプログラミング言語に移植する作業を考えてみてほしい。これについては、マイクロサービスへの移行が、技術の変更をより簡単にするかもしれない。その技術を必要とするマイクロサービス内

でのみ技術の変更を行い、残りの部分はそのままにしておくことで、変更の影響を軽減できるからだ。

2.2.4　堅牢性を改善する

シングルテナント型のソフトウェアからマルチテナント型のSaaSアプリケーションへの移行は、システム停止の影響が大幅に拡大することを意味する。生活におけるソフトウェアの重要性が高まるにつれ、ソフトウェアが常に利用可能であることの顧客の期待も高まってきている。そこで、アプリケーションを個々の独立してデプロイ可能なプロセスに分割することで、堅牢性を向上させるための多くの仕組みを可能にする。

マイクロサービスを使用することで機能が分解されるため、より堅牢なアーキテクチャを実装できる。つまり、機能のある領域に影響を与えても、システム全体をダウンさせる必要はなくなる。また、アプリケーションの堅牢性を最も必要とする部分に時間とエネルギーを集中させることで、システムの重要な部分が運用可能な状態を維持できるようになる。

回復性と堅牢性

発生した障害を円滑に処理して迅速に回復することで、システム停止を回避する能力を向上させたいとき、私たちはよく**回復性（レジリエンス）**を話題に上げる。現在、**レジリエンスエンジニアリング**として知られている分野では多くの研究が行われており、コンピューティングだけでなく、全ての分野に適用されるテーマとして扱われている。David Woodsによって開拓された回復性のモデルは、回復性の概念をより広く捉えている。特に、既知と未知の障害の原因に対処する能力を分離することによって、「回復性がある」という状態は私たちが最初に思い浮かべるほどには単純ではないという事実を指摘している[†3]。

David Woodsの同僚であるJohn Allspawは、堅牢性と回復性の概念を次のように区別して定義している。**堅牢性**とは、予想される変動に対応できるシステムを持つことを指す。そして、**回復性**とは、思いもよらないことに適応できる組織能力を持つことを指す。これには、カオスエンジニアリングのようなものを使って実験をする文化を作ることも含まれる。たとえば、特定のマシンが死ぬ可能性があることを認識しているなら、インスタンスの負荷分散を行うことでシステムに冗長性を持たせられる。これは堅牢性への取り組みの一例だ。回

復性とは、組織が全ての潜在的な問題を予測することはできないという事実に備えるプロセスだ。

[†3] David Woods, “Four Concepts for Resilience and the Implications for the Future of Resilience Engineering.” Reliability Engineering & System Safety 141 (2015) 5-9. [35]

ここで考慮すべき重要な点は、マイクロサービスがタダで堅牢性を与えてくれるわけではないということだ。むしろ、マイクロサービスは、ネットワークの分割やサービス停止などにいっそう耐えられるようなシステムの設計を必要とする。機能を複数の別々のプロセスや別のマシンに単に分散させただけでは、堅牢性の向上は保証されない。それどころか、単に障害の表面積を増やすだけになるだろう。

他にどんな方法があるか

モノリスの複数のコピーを、ロードバランサーやキューなどの負荷分散メカニズムの後ろで実行することで、システムに冗長性を追加できる[†4]。そして、全てのマシンを同じラックや同じデータセンターに置かないなど、モノリスのインスタンスを分散させて障害の影響を限定的にすることで、アプリケーションの堅牢性をさらに向上できる。

[†4] このような例の1つに、Gregor Hohpe、Bobby Woolfが執筆した“Enterprise Integration Patterns”（Addison-Wesley）[16]に記載されているCompeting Consumerパターンがある。

より信頼性の高いハードウェアとソフトウェアへの投資も、システム停止の既存の原因を徹底的に調査することと同様に、利益をもたらす可能性がある。たとえば、手動プロセスへの過度の依存や「約束事に従わない」人々によって引き起こされた本番環境での問題、これはしばしば個人による「悪気のない」ミスだが、そのような問題が重大な影響を及ぼすのを、私はこれまでに少なからず見てきた。ブリティッシュ・エアウェイズは2017年に大規模な障害に見舞われ、ロンドンヒースロー空港とガトウィック空港を発着する全便が欠航する事

態に陥った。この問題は、ある1人の個人の行動に起因する電力サージによって不用意に引き起こされたものだった。アプリケーションの堅牢性が誰もミスを犯さないことを前提としている場合には、あなたは不安定な道のりを行くことになるだろう。

2.2.5　開発者の数を増やす

多くの人が、プロジェクトをより速く進めるために開発者を投入した結果、それが裏目にでるという問題を目の当たりにしたことがあるだろう。しかし、問題によっては、解決のために多くの人員が必要な場合もある。今でも影響力を持つ書籍『人月の神話』（丸善出版）[6]の中でFrederick Brooksが語っているように、人員の増加がデリバリー速度の向上に寄与するのは、作業自体を相互作用が限定された別々の作業に分割できる場合だけだ。彼は畑で作物を収穫することを例に挙げている。収穫作業は複数の人が並行して作業するのは簡単な仕事だ。だが、ソフトウェアがこのように動作することはほとんどない。なぜなら、行われる作業が同じであることはなく、ある作業からの出力が別の作業の入力として必要とされることが多いからだ。

明確に識別された境界線と、マイクロサービス同士の結合を確実に制限することに焦点を当てたアーキテクチャによって、独立して作業できるコードの断片が手に入る。そうやって、デリバリー衝突を減らすことによって、開発者数の拡大が期待できる。

開発者の数をうまく拡大させるには、チームに自律性があることが求められる。マイクロサービスを所有しているだけでは十分ではない。チームとサービスの所有権がどのように揃っているか、チーム間でどのような調整が必要かを考えなければならない。また、あまりにも多くのサービスにまたがって変更を調整する必要がないように、作業を分割する必要もある。

他にどんな方法があるか

マイクロサービスは、それ自体が独立して作業可能な機能の断片となるため、大規模なチームにはうまく機能する。別のアプローチには、モジュラーモノリスがある。モジュラーモノリスでも、別のチームがそれぞれのモジュールを所有し、他のモジュールとのインターフェイスが安定している限りは、担当を分けて変更し続けられる。

ただし、このアプローチには限界がある。ソフトウェアは全て一緒にパッケージ化されるため、異なるチーム間で何らかの形で競合が起こる可能性があるからだ。そのため、デプロイの際には適切な関係者間の調整が必要になる。

2.2.6　新しい技術を受け入れる

モノリスはしばしば、技術的な選択肢を制限する。通常、バックエンドでは1つのプログラミング言語を使用し、1つのプログラミングイディオムを使用する。私たちは、1つのデプロイメントプラットフォーム、1つのオペレーティングシステム、1種類のデータベースに縛りつけられることになる。マイクロサービスアーキテクチャでは、サービスごとにこれらの選択を変えることが可能だ。

サービス境界で技術の変更を分離することで、新しい技術の利点を他と切り離して理解でき、その技術に問題があることが判明したとしても、その影響を制限できる。

私の経験からすると、成熟したマイクロサービス組織は、サポートする技術スタックの数を制限することがよくあるものの、使用する技術が均質であることはほとんどない。安全な方法で新しい技術を試せる柔軟性は、顧客にとってより良い結果を提供するという点でも、新しい技術を習得することで開発者を満足させ続けるという点でも、競争上の優位性をもたらす。

他にどんな方法があるか

ソフトウェアを単一のプロセスとしてデリバリーし続ける場合、導入できる技術には限界がある。もちろん、同じランタイムで新しい言語を安全に採用することは可能だ。同じ実行プロセス内で複数の言語で書かれたコードをホストできるランタイムには、たとえばJVMがある。ただし、新しいタイプのデータベースを導入することは、ずっと困難だ。段階的な移行を可能にするために、これまでのモノリシックなデータモデルに対して何らかの分解をする必要があるからだ。新しいデータベース技術に完全な即時切り替えを行う場合はその限りではないが、それはそれで複雑でリスクがある行為といえる。

現在の技術スタックが「燃えさかるプラットフォーム」[†5]とみなされている場合は、より新しく、よりサポートされている技術スタックに置き換える以外の選択肢はないかもしれない。もちろん、既存のモノリスを新しいモノリスに少しずつ置き換えていくことを止める理由は何もない。3章で説明する「パタ

ーン：ストラングラーアプリケーション」のアプローチは、そのためにうまく機能するだろう。

[†5] 「燃えさかるプラットフォーム（burning platform）」とは、北海上にある石油採掘プラットフォームが火事で燃えているときに、そこにいた作業員が30メートル下にある冷たくて死ぬかもしれない海に飛び込まざるをえなかったという逸話から転じて、環境を変えても生き抜けるかは不明なものの、留まるのは確実に死を意味するという場所や状態を指す用語。寿命を迎えた技術を表すのによく用いられる。そうした技術はサポートを得るのが難しすぎたり、高コストだったり、経験者を雇うのが難しかったりするからだ。ほとんどの組織が「燃えさかるプラットフォーム」だと考える技術の例には、COBOLメインフレームアプリケーションがある。

再利用？

再利用は、マイクロサービス移行のための最もよく言われる目標の一つだ。とはいえ、私の考えでは、それはそもそもあまり良い目標ではない。なぜなら、根本的に、再利用は人々が望む直接的な成果では**ない**からだ。再利用とは、人々が他の利益につながることを期待するものだ。人々が再利用によって期待するのは、フィーチャーをより早く出荷できるようになったり、コストを削減できるようになったりすることだ。もしそれらが目標であるなら、それらを追跡しよう。さもなければ、間違った最適化をしてしまうかもしれない。

このことを説明するために、再利用が目的に選ばれるよくある理由の1つを探ってみよう。私たちは、より迅速にフィーチャーを出荷したいと考えている。既存のコードを再利用することを中心に開発プロセスを最適化すれば、やるべき作業が減り、それほど多くのコードを書く必要がなくなると考えている。しかし、ここで簡単な例を見てみよう。Music Corpのカスタマーサービスチームは、顧客に請求書を発行するためにPDFをフォーマットする必要がある。システムの別の部分では、既にPDFの生成を処理している。倉庫での印刷用にPDFを生成したり、顧客に出荷された注文の梱包伝票を生成したり、サプライヤーに注文依頼を送ったりしている。

再利用という目標に沿って、私たちのチームは既存のPDF生成機能を使用するように指示されるかもしれない。しかし、その機能は現在、組織の別チームによって管理されている。そのため、自分たちのフィーチャーをサポートするの

に必要な変更を行うには、そのチームとの調整が必要になる。これは、そのチームに作業を依頼するか、自分たちで変更を行ってプルリクエストを出さなければならないことを意味する。いずれにしても、変更を行うには組織の別の部分と調整しなければならなくなる。

このように、他チームとの調整に時間を費やして変更を行い、それを公開していくこともできる。しかし、独自の実装を書いた方が、既存のコードを作り替えるのに時間を費やすよりもずっと早く顧客にフィーチャーを出荷できる。実際の目標が市場投入時間を短縮することであれば、その方が正しい選択だろう。しかし、市場投入時間の短縮を期待しながら再利用に向けて最適化してしまうと、結果的に速度を落としてしまう可能性がある。

複雑なシステムで再利用を評価するのは難しい。そして、すでに説明したように、再利用は通常、何か別のことを達成するために行うものだ。再利用が常に正しい答えとは限らないことを認識して、実際の目的のために時間を費やそう。

2.3　マイクロサービスが悪いアイデアのとき

ここまでは、時間をかけてマイクロサービスアーキテクチャの潜在的な利点を探ってきた。しかし、いくつかの状況ではマイクロサービスを全く使わないことをお勧めしている。ここでは、そのようないくつかの状況を見ていこう。

2.3.1　不明瞭なドメイン

サービス境界の誤りは高くつく可能性がある。サービス境界を間違えると、サービスをまたぐ変更の数が増えたり、コンポーネントが過度に結合されたりして、一般的には、単一のモノリシックシステムを持つよりも悪い結果になる可能性がある。前著『マイクロサービスアーキテクチャ』[25]では、その事例としてThoughtWorksのSnapCI製品開発チームの経験を紹介した。SnapCI製品開発チームは、継続的インテグレーションの領域を熟知していた。それにもかかわらず、ホスト型CIソリューションのサービス境界を考える際の最初の試みは、全く適切ではなかった。そのため、変更コストと所有コストが高くついてしまっていた。数カ月間その問題と格闘した後で、チームはサービスを1つの大きなアプリケーションに統合することを決定した。その後、アプリケーションのフィーチャーセットがある程度安定してきて、チームがドメインをより深く理解するようになると、それらに対する安定した境界線は容易に見つけられた。

SnapCIは、ホスト型の継続的インテグレーション（CI）および継続的デリバリー（CD）のツールだった。チームは過去に、同様のツールであるGoCDも担当していた。GoCDは、現在はオープンソースの継続的デリバリーツールで、クラウドでホストされるのではなく、ローカルにデプロイ可能なツールだった。SnapCIとGoCDプロジェクトの間では、初期の段階でコードの再利用があったが、最終的にはSnapCIは全く新しいコードベースになった。とはいえ、CDツールドメインでの過去の経験は、サービスの境界を特定し、マイクロサービスの集合としてシステムを構築できるようチームを後押しした。

しかし、数カ月後には、SnapCIのユースケースが微妙に異なることが明らかになり、サービス境界に対する最初の考えが正しくないことが分かった。そのため、サービス間で多くの変更が行われ、それに伴う変更コストが高くなっていた。最終的に、チームはサービスを1つのモノリシックシステムに統合し、境界線がどこに存在すべきかをよりよく理解するための時間を確保した。1年後、チームはモノリシックシステムをマイクロサービスに分割でき、その境界線ははるかに安定していることが証明された。このような事例を、私は多く見てきている。システムを早々にマイクロサービスに分解すると、特にそのドメインに慣れていない場合には、コストがかさんでしまうことがある。多くの点で、既存のコードベースをマイクロサービスに分解することは、最初からマイクロサービスに向かおうとするよりもはるかに簡単だ。

自分のドメインをまだ完全に把握できていないと感じているのであれば、システムの分解に着手する前にそれを解決しておくのも良いアイデアだろう（これがドメインモデリングを行うもう1つの理由だ。後ほど手短に説明する）。

2.3.2　スタートアップ

マイクロサービスを利用していることで有名な組織の多くがスタートアップ企業とみなされているのになぜ？と思うかもしれない。だが、実際にはNetflixやAirbnbなどの多くの企業がマイクロサービスアーキテクチャに移行したのは、その進化の後半になってからだ。マイクロサービスは、「スケールアップ」と呼ばれるスタートアップ企業、つまり、製品／市場フィットの基礎を少なくとも確立し、収益性を高めるために（あるいは単に利益をあげるために）スケーリングを行っている企業にとっては、素晴らしい選択肢となるものだ。

スタートアップ企業は、そうしたスケールアップ企業とは異なり、製品／市場フィットを見いだすために、多くの場合、さまざまなアイデアを試している段

階だ。この段階では、探索が進むにつれて製品の元となるビジョンが大きく変わる可能性があり、それは製品ドメインにも大きな変化をもたらす。

実際のスタートアップとは、資金が限られている小規模な組織であることが多い。そうした企業は製品に適した市場を見つけることに全力を注ぐ必要がある。マイクロサービスは、顧客ベースに適合するものを見つけた後でスタートアップが抱える問題を主に解決する。言い方を変えれば、マイクロサービスはスタートアップとして最初の成功に達した後に発生する問題を解決するための素晴らしい方法といえる。だから、最初は成功することに集中しよう！　そもそものアイデアが悪かったなら、マイクロサービスで構築するかどうかは重要ではない。

スタートアップが作るような新規の「グリーンフィールド」[†6]システムで最初からそうするよりも、既存の「ブラウンフィールド」システムを分割する方がはるかに簡単だ。何せやれることが多くある。調べられるコードがあるし、システムを使用している人や保守している人に実際に話を聞ける。あなたは「良い」とはどのようなものかも知っている。つまり、変更作業を行うためのシステムを持っているので、意思決定プロセスでミスをしたり、積極的になりすぎたりした場合には、それを簡単に検知できる。

[†6] 訳注：建設業からのアナロジー。グリーンフィールドとは汚染されていない未開発な土地を指す。再開発または再利用しようとしている、既存汚染物質を伴う用地のことはブラウンフィールドと呼ばれる。

また、実際に稼働しているシステムも持っている。システムがどのように動作し、本番環境でどのように動作するかを理解している。マイクロサービスに分解すると、厄介なパフォーマンスの問題が発生する可能性があるが、ブラウンフィールドシステムでは、パフォーマンスに影響を与える可能性のある変更を行う前に、健全なベースラインを確立するチャンスがある。

「スタートアップは絶対にマイクロサービスを使うな」と言っているわけではない。ここで言いたいのは、これらの要因にもとづいて慎重になるべきだということだ。初めは境界線がはっきりしている部分だけを分割し、それ以外の部分はよりモノリシックな側に置くようにしよう。これはまた、運用の観点から自分の成熟度を評価する時間を確保することでもある。もし2つのサービスの

管理に苦労しているのであれば、10のサービスを管理するのは到底不可能だろう。

2.3.3　顧客の環境にインストールして管理するソフトウェア

顧客自身が運用を行うようなソフトウェアを開発し、出荷している場合には、マイクロサービスは悪い選択かもしれない。マイクロサービスアーキテクチャに移行すると、運用領域に多くの複雑さが押し寄せてくる。モノリシックなシステムのデプロイ監視やトラブルシューティングに使用していた以前の手法が、新しい分散システムでは機能しない可能性がある。現在、マイクロサービスへの移行を行うチームは、新しいスキルを身につけたり、新しいテクノロジーを採用したりすることで、これらの課題を相殺している。これらの課題は、一般的には最終顧客に期待できるものではない。

通常、顧客がインストールして使用するソフトウェアは、特定のプラットフォームをターゲットにする。たとえば、「Windows 2016 Serverが必要」「macOS 10.12 以上が必要」といった具合だ。これらはターゲットとするデプロイ環境が明確に定義されており、これらのシステムを管理する人々がよく理解しているメカニズムを使用してモノリシックソフトウェアをパッケージ化している可能性が高い（たとえば、Windowsインストーラーパッケージにバンドルされた Windowsサービスを出荷するなど）。顧客は、このような方法でソフトウェアを購入して実行することに慣れている可能性が高い。

あなたが顧客に提供しているソフトウェアが、1つのプロセスを実行・管理するものから、10から20個のプロセスを管理・実行するものに変わった場合のトラブルを想像してみてほしい。あるいはもっと積極的に、Kubernetesクラスタなどでソフトウェアを実行することを期待するようになったとすると、どんな問題が起こりそうだろうか。

顧客がマイクロサービスアーキテクチャを管理するスキルやプラットフォームを持っていることは期待できないのが現実だ。仮に持っていたとしても、あなたが必要とするスキルやプラットフォームを持っていないかもしれない。たとえば、Kubernetesのインストール一つとっても、そのスキルには大きなばらつきが出るだろう。

2.3.4　もっともな理由を持たないとき

そして最後に、マイクロサービスを採用してはならない最大の理由がある。それは、何を達成しようとしているのかを明確に把握していない場合だ。これから説明するように、これから説明するように、**どこから**移行を開始し、システムを**どのように**分解するかは、何を求めてマイクロサービスを採用するかによって変わってくる。目標の明確なビジョンがなければ、暗闇の中で手探りすることになる。みんながやっているからという理由だけでマイクロサービスに手を出すことは、絶対にしてはいけない。

2.4　トレードオフ

ここまでは、人々がマイクロサービスを採用したいと思う理由をそれぞれ示し、他の選択肢を（簡単にだが）検討してきた。しかし、現実の世界では、人々はある一つのことだけを変えるのではなく、多くのことを一度に変えようとするのが常だ。そのことは、優先順位を混乱させ、必要な変化の量をすぐに増やしてしまい、何らかの効果を得るのを遅らせてしまう可能性がある。

たいていのことは無邪気にはじまる。トラフィックの大幅な増加に対応するためにアプリケーションを再設計する必要があり、マイクロサービスが今後進むべき道であると判断したとする。そうすると、他の誰かが出てきてこう言う。「マイクロサービスを使えば、チームの自律性を高められますよ！」 また別の誰かはこう言う。「これはプログラミング言語としてKotlinを試す絶好のチャンスです！」 そうこうするうちに、チームの自律性を高め、アプリケーションの規模を拡大し、新しい技術を一度に導入しようとし、おまけに人々が計画表に付け加えたものまで含んだ、大規模な変更のイニシアチブを握ってしまうことになる。

さらに、このような状況では、マイクロサービスは**そのため**のアプローチとして固定化されてしまう。もしあなたがスケーリングの面だけに焦点を当てているなら、移行の間に、既存のモノリシックアプリケーションを水平方向にスケーリングした方が良いことに気付くかもしれない。しかし、そうすることで、チームの自律性を向上させたり、プログラミング言語としてKotlinを導入したりという新たな副次的な目標を達成することはできなくなってしまう。

したがって、移行の背後にある核となる動機と、達成できるとよいかもしれない二次的な利益を切り離すことが大切となる。上記の例であれば、アプリケーションのスケールを改善することが最も重要だ。他の副次的な目標（チームの

自律性の向上など）を達成するために行われる作業は有用かもしれないが、それらが邪魔になったり、主要な目標から遠ざかったりする場合は、後回しにすべきだ。

ここで大切なことは、いくつかの物事が他の物事よりも重要だと認識することだ。そうでなければ、適切に優先順位をつけることはできない。何が重要かを認識する作業で私が好きなのは、それぞれの望ましい成果をスライダーとして考えることだ。各スライダーは真ん中から始まる。何かをより重要にする場合には、他の優先順位を下げなければならない。例を図2-1に示す。ここでは、たとえば、多言語プログラミングをより簡単にしたいと思っていても、それはアプリケーションの耐障害性を向上させることよりは重要でないと明確に示されている。こうした結果を明確に順位付けしておけば、今後どのように進めていくかを考える際に意思決定をより簡単に行える。

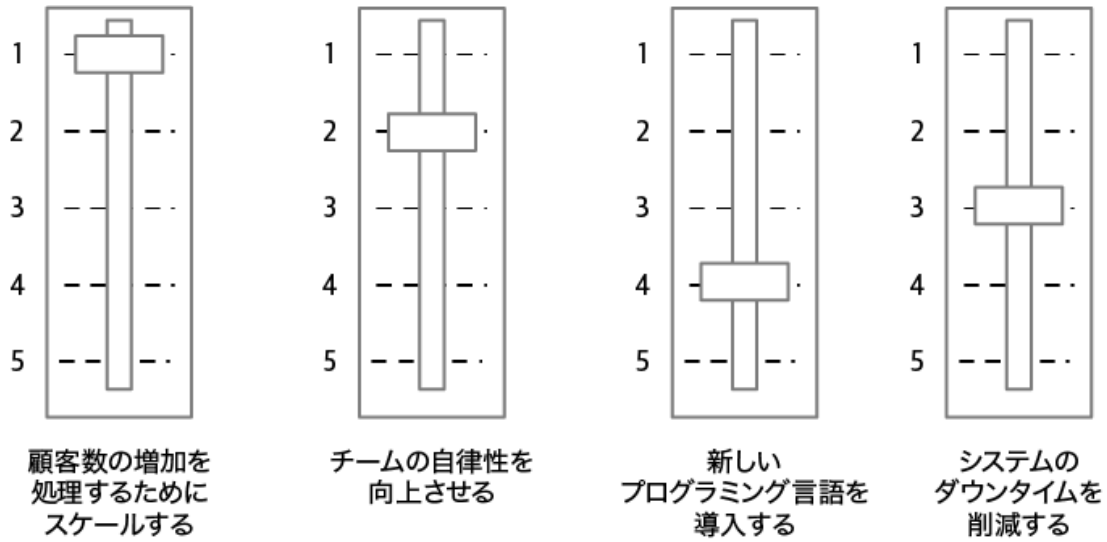

図2-1　スライダーを使用して競合する優先順位のバランスを取る

こうした相対的な優先順位は変化する可能性がある（そして、それはより多くのことを学ぶにつれて変化するはずだ）。しかし、このスライダーは意思決定の指針となる。責任を分散させ、新たに自律的になったチームにより多くの権限を渡したい場合、このようなシンプルなモデルは、チームが行う意思決定に情報を与え、組織全体で達成しようとしていることに沿ったより良い選択をするのに役立つ。

2.5　みんなを連れていく

よく受ける質問に「マイクロサービスを上司に売り込むにはどうしたらいいですか？」というものがある。この質問は、マイクロサービスアーキテクチャの

可能性を知り、それが前進する道だと確信している開発者から寄せられることが多い。

あるやり方への同意が得られないとき、多くの場合は、あなたが達成したいことについて、その人たちは異なる見解を持っている。重要なのは、あなたと一緒に旅に出る必要のある人たちと、あなたが**何**を達成しようとしているかについての共通理解を持つことだ。もしそれについて問題ないなら、少なくとも**どうやって**そこに到達するかについての意見だけが異なっているのだと分かる。その場合には、再び目標に立ち返って考えてみよう。もし組織内の他の人々と目標を共有できているのであれば、変革に参加してもらえる可能性はずっと高くなる。

組織変革を支援するためのよく知られたモデルから発想を得て、アイデアを売り込んで実現する方法を探るのは、やってみる価値があることだ。次にそれを見ていこう。

2.6　組織を変革する

John Kotterによる組織変革を実施するための8段階のプロセスは、世界中のチェンジマネージャーにとって定番のモデルだ。この優れたモデルは、組織変革に必要な活動を独立した分かりやすい手順にしたものだ。これが組織変革のための唯一のモデルというわけではないが、私はもっとも役立つモデルの1つだと考えている。

モデルの概要を図2-2に示す。プロセスの説明は豊富に存在するため、ここでは詳細についてはあまり触れない[†7]。しかしながら、各手順を簡単に説明し、マイクロサービスアーキテクチャを検討する際にそれらがどのように役立つかを考えることには価値がある。

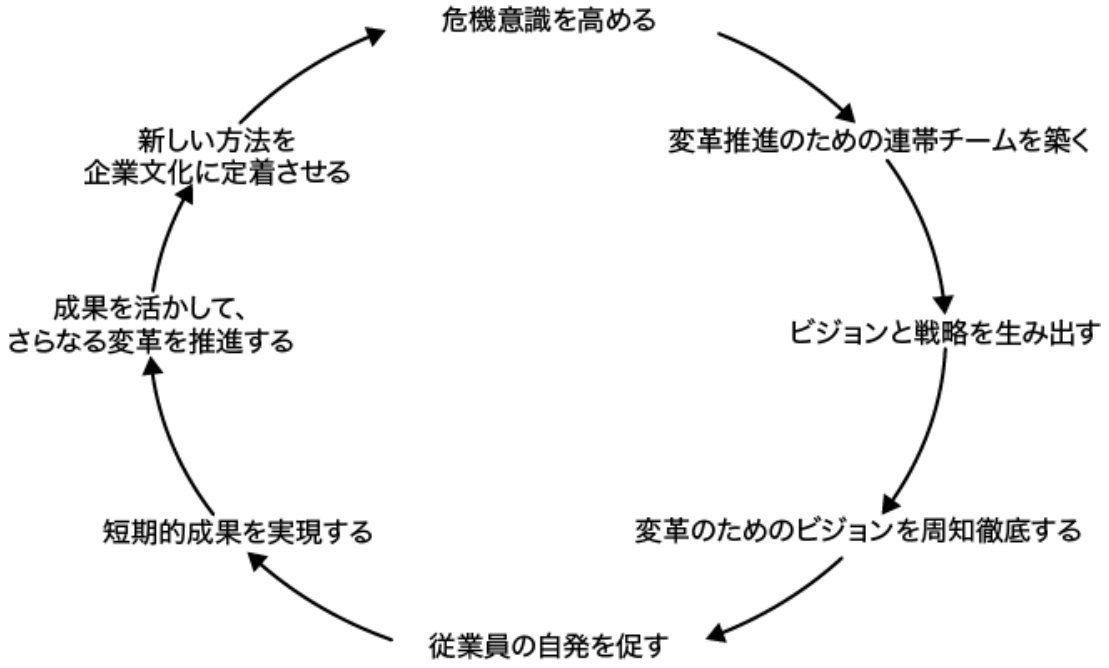

図2-2　John Kotterの組織変革のための8段階のプロセス

[†7] John Kotterの変革モデルについては彼の著作『企業変革力』（日経BP）[22]で詳しく解説されている。

プロセスの概要を説明する前に注意しなければならないことがある。この変革モデルは通常、大規模な組織行動の変革を引き起こすために使用されるものだ。したがって、10人のチームにマイクロサービスを導入しようとしているだけであれば、このモデルは大げさかもしれない。しかし、私の経験では、そうした小さなスコープの変更であっても、特に初期のステップで、このモデルは有用であった。

2.6.1　危機意識を高める

マイクロサービスに移行するというアイデアは、良いアイデアだと思ってもらえるかもしれない。問題は、あなたのアイデアは、組織の周囲にたくさんある良いアイデアの一つにすぎないということだ。そこで、**今こそ**この特定の変更を行うべき時だということを理解してもらうことが重要となる。

このステップで役に立つのは「教育可能な」瞬間を探すことだ。「火事の後の火の用心」という言葉の通り、火事の後こそ、それにうってつけのタイミングだ。なぜなら、火事が起きたことで、問題に意識を向ける必要が生じ、あらかじめ用心をしておくことでそれを回避できると認識するからだ。危機を乗り切

った直後こそ、変革を推進する方へ人々の意識を後押しできる瞬間となる。ゆっくりしていると、痛みやその原因に対する意識は薄れていってしまう。

覚えておこう。あなたがすべきことは「今すぐマイクロサービスをやるべきだ！」と主張することではない。達成したいことについての危機意識を共有することだ。そして、既に述べたように、マイクロサービスは目標ではないということを忘れてはいけない。

2.6.2　変革推進のための連帯チームを築く

全員を巻き込む必要はないものの、変革を実現するには十分な人数が必要だ。あなたの組織の中で、この変化を推進する手助けをしてくれる人物を見つける必要がある。あなたが開発者であれば、チーム内の直属の同僚か、技術リーダーやアーキテクト、デリバリーマネージャーといったもっと上役の誰かから探し始めることになるだろう。変革の範囲にもよるが、たくさんの人を巻き込む必要はないかもしれない。チームのやり方を変えるだけであれば、時間をたっぷり確保するだけで十分かもしれない。自社で開発されているソフトウェアの開発方法を変えようとしているのであれば、それを推進する役員レベルの誰か（おそらくCIOやCTO）がこれを擁護する必要があるだろう。

この変革を支援するために人を集めるのは簡単ではないかもしれない。どんなに良いアイデアだと思っていても、あなたのことを聞いたことがなかったり、一緒に仕事をしたことがなかったりしたら、その人があなたのアイデアを支持してくれることは難しいだろう。信頼は得るものだ。既にあなたと仕事をし、マイクロサービス化よりは小さな、手早く得られる成果をともに上げたことのある人であれば、あなたの大きなアイデアを支持してくれる可能性が高くなる。

このステップで重要なのは、ソフトウェアデリバリーに関わっていない人々を巻き込むことだ。もしあなたが既に「IT」と「ビジネス」の障壁が取り払われた組織で働いているのであれば、これはおそらく問題ないだろう。一方で、そうしたサイロがまだ存在する場合は、他の場所で支援者を見つけるために、通路から身を乗り出して手を伸ばす必要があるかもしれない。もちろん、マイクロサービスアーキテクチャの採用がビジネスが直面している問題を解決することに焦点を当てているのであれば、これはもっと簡単に売り込めるだろう。

ITサイロ外の人々の関与が必要な理由は、あなたの行う変更の多くが、ソフトウェアの機能や振る舞いに大きな影響を与える可能性があるからだ。たとえ

ば、障害モード時のシステムの動作方法や、レイテンシーへの取り組み方など、さまざまなトレードオフを行う必要がある。たとえば、サービス呼び出しを回避するためにデータをキャッシュしておくことは、分散システムの主要な操作のレイテンシーを確実に短縮するための良い方法だが、システムのユーザーが古いデータを見ることになりかねないというトレードオフがある。それは正しいことなのだろうか？　この点については、おそらくユーザーと話し合う必要があるだろうが、組織内でユーザーのニーズを把握している人たちが変更の背景にある合理性を理解していない場合は、難しい話し合いになるだろう。

2.6.3　ビジョンと戦略を生み出す

このステップでは、仲間を集めて、どのような変化をもたらすことを望んでいるのか（**ビジョン**）と、そこに到達する方法（**戦略**）をまとめる。それらは現実的でありながら意欲的である必要があり、この2つの間のバランスを見極めることが鍵となる。ビジョンがより広く共有されればされるほど、それをパッケージ化して人々を巻き込むための作業が必要になる。しかし、小規模なチームであれば、ビジョンは漠然としたもの（「バグの数を減らさなければならない！」など）であっても効果はあるだろう。

ビジョンとは主に目標に関するもの、つまり、**何**を目指すかだ。そして、戦略とは**どのように**それを行うかだ。マイクロサービスはその目標を達成するためのものだ（マイクロサービスが戦略の一部になることがあなたの願いだ）。しかし、戦略は変化する可能性があることを忘れてはいけない。ビジョンにコミットすることは重要だが、反証に直面してもなお特定の戦略に過度にコミットするのは危険だ。それは、重大なサンクコストの錯誤につながる可能性がある。

2.6.4　変革のためのビジョンを周知徹底する

大きなビジョンを持つことは素晴らしい。しかし、人々が可能だと信じられないような大きなビジョンにしてはいけない。最近、ある大きな組織のCEOが出した次のような声明を見かけた。

> *我々は、今後12カ月で、マイクロサービスへの移行やクラウドネイティブ技術の導入によって、コスト削減とより迅速な価値提供を実現したいと考えている。*
>
> *匿名のCEO*

私が聞いた限りでは、その企業のスタッフは誰一人として、このようなことが可能だとは思っていなかった。前述のステートメントには、潜在的に矛盾した目標が提示されているという問題がある。ソフトウェアのデリバリー方法を全面的に変更することで、より迅速な対応が可能になるかもしれない。しかし、そのためには新しいスキルを持ち込む必要があるだろうから、12カ月以内にはコストは削減されないだろう。もう一つの問題は、ここで示されている期間だ。この組織の変化のスピードは、12カ月という目標が冷笑される程度には遅かった。この例から分かるように、どのようなビジョンであっても、それがある程度信じられるものでなければならない。

ビジョンの共有は小さく始められる。何年も前、私はGoogleでテスト自動化のプラクティスを展開するための「Test Mercenaries」と呼ばれるプログラムに参加していた。このプログラムは、今ではコミュニティ・オブ・プラクティスと呼ばれている集まり（実践コミュニティともいう。Googleの命名法では「グループレット」）が、自動テストの重要性を共有するためにもともと取り組んでいたものから始まった。プログラムにおけるテストに関する情報を共有するための初期の取り組みの1つは「トイレでのテスト」と呼ばれる取り組みだった。これは、トイレのドアの裏に1ページの短い記事を貼り付けて、人々が「暇つぶしに」読めるようにしたものだ。この手法がどこでも通用するとは言わないが、Googleではうまく機能していたし、小さくて実行可能なアドバイスを共有するための本当に効果的な方法だった[†8]。

[†8] Googleでテストを広めるための変革戦略の詳細な歴史については、Mike Blandの素晴らしい記事 “Test Mercenaries”（http://bit.ly/2omkxVy）[3]が一見の価値がある。Mikeは「トイレでのテスト」の詳細な歴史についても “Testing on the Toilet”（http://bit.ly/2ojpWwm）[4]で解説している。

最後に注意点を一つ。最近では、Slackのようなシステムを利用して、対面でのコミュニケーションから離れていく傾向が強まっている。この種の重要なメッセージを共有する場合は、対面でのコミュニケーション（理想的には対面だが、ビデオでもよいだろう）の方がはるかに効果的だ。このようなアイデアを聞いた人々の反応を理解しやすくなり、メッセージを調整したり誤解を避けるのにも役立つ。大きな組織全体にビジョンを伝えるために他のコミュニケーション手段が必要になったとしても、まずはできるだけ多くの人に会って話をしてみよう。そうすることで、より効率的にメッセージを洗練させられる。

2.6.5　従業員の自発を促す

変革を支援する経営コンサルタントは、「従業員に権限を渡しましょう」とよく言う。多くの場合、これはとても単純なこと、つまり障害を取り除くことを意味する。

ここまでで、ビジョンを共有し、ワクワク感を高めた。次は、何が起こるのだろうか？　何か邪魔が入る。最も一般的な問題の1つは、人々が今やっていることに忙しすぎて、変化を起こす余裕がないということだ。これが、追加のゆとりと専門知識をチームに与えようと、企業が組織に（おそらく採用やコンサルタントとして）新しい人材を雇い入れようとするよくある理由だ。

具体例を挙げよう。マイクロサービスの導入に関しては、インフラストラクチャのプロビジョニング周りの既存プロセスが実際の問題になる可能性がある。本番環境へ新しいサービスのデプロイするために半年前にハードウェアを発注する必要がある場合、仮想化された実行環境（仮想マシンやコンテナなど）をオンデマンドでプロビジョニングできる技術を採用することで、パブリッククラウドベンダーへの移行と同様に、大きな恩恵がもたらされる可能性がある。

だがしかし、前の章でのアドバイスをここでも繰り返そう。新しい技術は、そのためだけに取り入れるのではなく、具体的な問題を解決するために取り入れるべきだ。新しい技術は目に見える具体的な問題を解決するために導入しよう。障害を特定したら、それらの問題を解決するために新しい技術を取り入れるのだ。完璧なマイクロサービスプラットフォームを定義するために1年を費やすというような罠に陥ってはならない。それは、実際にあなたが抱えている問題を解決しないことを確認するだけの結果となる。

GoogleのTest Mercenariesプログラムの中で、私たちは、テストスイートの作成と管理を容易にするためのフレームワークを作成し、コードレビューシステムの一部としてテストの可視化を促進し、さらにはテストの実行を容易にするための新しい全社的なCIツールの作成を推進した。しかし、私たちはこれを一度に全部やったわけではない。いくつかのチームと協力して問題点を確認し、そこから学び、新しいツールを導入するために時間を投資した。また、テストスイートの作成を容易にするというのは非常に単純なプロセスだったが、全社的なコードレビューシステムを変更するというのは、もっと大きな課題だった。他の場所でいくつかの成功を収めるまでは、それを試さなかった。

2.6.6　短期的成果を実現する

人々が進歩に気付くまでに時間がかかりすぎると、ビジョンへの信頼は失われてしまう。そのため、迅速な成果を目指そう。小さくて簡単で、実現しやすい成果にまず焦点を当てることで、勢いをつけられる。マイクロサービスへの分解に関しては、モノリスから簡単に抽出できる機能をリストの上位に入れるべきだ。しかし、これまでに述べてきたように、マイクロサービス自体が目的ではない。そのため、機能の一部を抽出する容易さと、それによってもたらされる利益のバランスを取る必要がある。この考え方については、章の後半で改めて触れる。

簡単だと思うことを選んだはずが、その実現に大きな問題を抱えてしまったとしても、それは戦略への貴重な示唆になる。もしかすると、取り組んでいることについて考え直させられるかもしれない。しかし、そのことは全く問題ない。重要なことは、最初に簡単なことに焦点を当てるならば、**早い段階で**この示唆を得られる可能性が高いということだ。私たちにできることは、一刻も早くミスから学べるように、物事を構築することだ。

2.6.7　成果を活かして、さらなる変革を推進する

ある程度の成功が得られたら、次に重要なのはその栄光にとらわれないようにすることだ。努力を続けなければ、短期的な勝利が唯一の勝利になってしまうかもしれない。成功（と失敗）の後は、いったん立ち止まってふりかえることが重要だ。そうすることで、変化をどのように推進し続けるかを考えられる。組織のさまざまな部分に手が届くように、アプローチを変える必要があるかもしれない。

マイクロサービスへの移行は、より深く切り込んでいくと、難しさが増す場合がある。データベースの分解を扱うことは、最初のうちは後回しにしていられるかもしれないが、いつまでも先延ばしにしておくことはできない。4章で説明するように、これには多くのテクニックがあるが、どれが正しいアプローチなのかは慎重に考える必要がある。モノリスのある領域でうまくいったからといって、その分解テクニックが他で通用しないことだってある。そうしたことを肝に銘じながら、前進するための新しい方法を常に試していく必要がある。

2.6.8　新しい方法を企業文化に定着させる

イテレーションを続け、変更を公開し、成功（と失敗）のストーリーを共有していくことで、新しい取り組み方は普通の仕事になり始める。このステップで行うことの大部分は、同僚や他のチーム、組織内の他の人たちとのストーリーの共有だ。難しい問題を解決した後に、すぐ次の問題に移ってしまうことがよくある。変化をスケールアップして定着させるためには、組織内で情報を共有する方法を継続的に探すことが不可欠だ。

時間がたつにつれて、新しいやり方が当たり前のやり方になっていく。マイクロサービスアーキテクチャを採用している企業であれば、それが正しいアプローチであるかどうかは問題ではなくなっていく。それが今のやり方であり、組織はそのやり方をよく理解している。

これは、ひいては新たな問題を生み出すことになる。あなたの新しくてすごいアイデアが確立された作業方法になったとして、もっと良いアプローチが姿を現せるように、さらにいうなら既存のものに取って代われるようにするには、どうすればよいだろうか？

2.7　段階的に移行していくことの重要性

> *ビッグバンリライトをしても、保証されるのはビッグバンだけだ。*
>
> *Martin Fowler*

既存のモノリシックシステムを分解することが適切だと判断した場合には、そのモノリシックシステムを少しずつ解体していくことを強くお勧めする。段階的なアプローチは、マイクロサービスについて学びながら進めていくのに役立つし、何かを間違えることによる影響を最小限に抑えられる（間違えることだってもちろんあるだろう！）。モノリスを大理石の塊だと考えてほしい。それを丸ごと吹き飛ばすこともできるが、それでうまくいくことはほとんどない。少しずつ削っていく方がはるかに理にかなっている。

問題は、重要なモノリシックシステムをマイクロサービスアーキテクチャに移行させるための実験コストが大きくなることだ。しかし、一度に全てを行うとすると、どれがうまくいっているのか（あるいはうまくいっていないのか）、良いフィードバックを得るのは難しくなってしまう。このような旅は小さな段階に分けた方がずっと簡単だ。それぞれを分析し、そこから学べるからだ。ア

ジャイルが登場する以前から私が漸進的なソフトウェアデリバリーの大ファンであり続ける理由がそれだ。私たちには、失敗を犯すことを受け入れ、その失敗の大きさを減らす方法が必要なのだ。

マイクロサービスアーキテクチャへの移行は、これらの原則を念頭に置く必要がある。大きな旅路をたくさんの小さなステップに分割しよう。それぞれのステップは実施から学べるものになるだろう。もしそれが逆行するステップであることが判明したとしても、それは小さなステップにすぎない。いずれにしても、あなたはそこから学び、次のステップはその前のステップから情報を得られることになる。

先ほども述べたように、物事をより小さな断片に分割することで、早い段階で勝利を見極め、そこから学べるようになる。これは次のステップを容易にし、勢いをつけるのに役立つ。また、マイクロサービスを1つずつ分割することで、大規模なデプロイを待つのではなく、マイクロサービスがもたらす価値を少しずつ引き出していける。

これらは全て、マイクロサービスを検討している人たちに向けた、ほとんどお決まりの次のアドバイスにつながる。マイクロサービスが良いアイデアだと思うのであれば、どこか小さなところから始めてみよう。機能の1つか2つの分野を選び、それをマイクロサービスとして実装し、本番環境にデプロイしてみて、それがうまくいったかどうかをふりかえってみよう。この章の後半では、どのマイクロサービスから始めるべきかを見極めるためのモデルを紹介する。

2.7.1　本番環境が重要

本番環境に反映されて積極的に利用されるようになるまでは、マイクロサービスの抽出が完了したとはいえないと肝に銘じることが**本当に**大切だ。段階的な抽出を行う目的の1つは、分解そのものから学び、その影響を理解する機会を得ることにある。重要な教訓の大部分は、サービスが本番になるまでは学ぶことはできない。

マイクロサービスの分解は、トラブルシューティング、フロートレーシング、レイテンシー、参照整合性、カスケード障害といった多くの問題を引き起こす可能性がある。これらの問題のほとんどは、本番稼働後に初めて気付くものだ。次の章からは、本番環境へのデプロイを可能にしながらも、問題が発生したときの影響を制限するテクニックを見ていく。ちょっとした変更を加えるだけで、自分で作成した問題を発見（修正）するのがはるかに簡単になる。

2.8　変更のコスト

本書では小さく段階的な変更を推奨している。それには様々な理由があるが、重要な原動力の一つは、私たちが行う各変更の影響を理解し、必要に応じて軌道修正を行うことにある。それによって、失敗のコストをより軽減できる。とはいえ、誤りの可能性を完全には排除できるわけではない。私たちは間違いを犯す可能性があるし、その事実を受け入れなければならない。それでも、そうした誤りのコストをどのように軽減するのが最善かを、私たちは理解しなければならない。

2.8.1　可逆的な判断と不可逆的な判断

AmazonのCEOであるジェフ・ベゾスは、毎年の株主レターでAmazonの仕組みについて興味深い考察を提供している。2015年のレターには、次のような優れた考察があった。

> *決定には、結果的に不可逆的なものやほぼ不可逆的なものがあります。これらは一方通行のドアのようなものです。こうした決定を行う際は、慎重に、綿密に、ゆっくりと、熟慮と協議を重ねて行われなければなりません。通り抜けて反対側に出た後で見えているものが気に入らなかったとしても、以前の場所には戻れないからです。これをタイプ1の決定としましょう。しかし、ほとんどの決定はそのようなものではなく、変更可能で、可逆的で、双方向のドアです。もしあなたがこうしたタイプ2の決定で最適でない決定をしてしまったとしても、その結果に長く耐える必要はありません。扉を開けて、また戻ればいいのです。タイプ2の決定は、判断力の高い個人や少人数のグループが迅速に行えますし、そうすべきです。*
>
> *ジェフ・ベゾス、Amazon株主への手紙（2015）*

ベゾスは、決定を行わない人は、タイプ2の決定をタイプ1の決定と同じように扱うという罠に陥ってしまうことが多いと述べている。全てが生死に関わることになり、全てが大きな仕事になってしまうのだ。マイクロサービスアーキテクチャを採用することの問題は、物事をどのように行うかについての選択肢が増えてしまうことにある。つまり、これまで以上に多くの決定をしなければならなくなる可能性があるということだ。もしあなたやあなたの組織が決定を行うことに慣れていなければ、この罠に陥ってしまい、進歩が止まってしまう可能性がある。

タイプ1やタイプ2といった用語はあまり説明的ではなく、実際に何を意味するのかを覚えるのは難しい。そこで、私はタイプ1を**不可逆的**、タイプ2を**可逆的**と呼ぶことにしている[†9]。

[†9] この命名のヒントをくれたMartin Fowlerに敬意をしめす。

私はこのコンセプトを気に入っているが、意思決定が常にこのどちらかのバケツにすっきりと収まるとは考えていない。むしろ、図2-3に示すように、「不可逆性」と「可逆性」はスペクトルの両端にあると考えている。

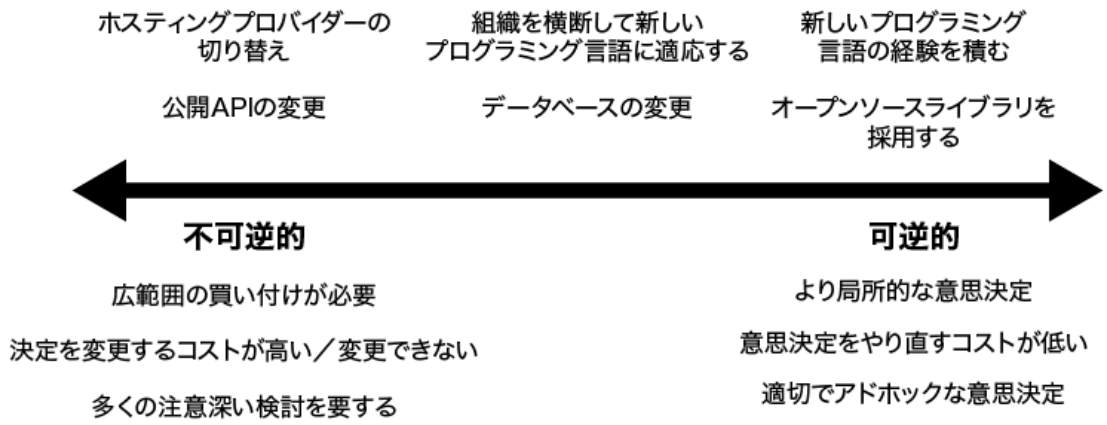

図2-3　スペクトルに沿った例で示す不可逆的決定と可逆的決定の違い

自分がこのスペクトルのどこにいるかを評価するのは、最初は難しいかもしれない。しかし、基本的には、後で気が変わった場合の影響を理解することで、それを評価できる。後からの軌道修正の影響が大きければ大きいほど、それは不可逆的な決定だとみなせる。

実際には、マイクロサービスへの移行の中で行う決定の多くは、「可逆的」と呼ばれるものになる。ソフトウェアには、ロールバックや取り消しが可能な特性がある。たとえば、ソフトウェアの変更やデプロイはロールバックできる。考慮しなければならないのは、後で気が変わったときのコストだ。

不可逆的な決定には、より多くの関与、慎重な検討、考慮が必要であり、（当然のことながら）それらの上でより多くの時間を取る必要がある。可逆的な決定、つまりこのスペクトルの右に行けば行くほど、問題に詳しい同僚に正しい判断を委ねられる。もし彼らが間違った決定をしたとしても、後から修正するのは容易だと分かっているからだ。

2.8.2　試しやすい場所

コードベース内でコードを移動させるコストはかなり小さい。それをサポートするツールはたくさんあるし、問題が発生しても修正は通常迅速に行える。それに比べ、データベースの分割ははるかに大変な作業であり、変更を切り戻すのもずっと複雑な作業となる。同様に、サービス間で過度に結合された統合をほどいたり、複数のコンシューマーが使用するAPIを完全に書き換えたりすることは、大規模な作業になる可能性がある。変更にかかるコストが大きいということは、これらの作業がますますリスクを伴うことを意味している。このリスクをどのように管理すればよいのだろうか？　それに対する私のアプローチは、影響が最も小さくなるところでミスをすることだ。

私は、変化のコストと間違いのコストが可能な限り低い場所、つまりホワイトボードで思考の多くを行う傾向がある。提案されたデザインをスケッチしよう。サービスの境界線を越えてユースケースを実行するとどうなるかを見てみよう。たとえば、Music Corpの場合であれば、顧客がレコードを検索したり、Webサイトに登録したり、アルバムを購入したりしたときに何が起こるかを想像してみてほしい。どのような呼び出しが行われるだろうか？　奇妙な循環型の参照が起こり始めるだろうか？　2つのサービスが通信しすぎていて、それらが1つのものであるべきということを示していないだろうか？

2.9　どこから始めればよいか？

さて、ここまで、目標を明確にし、存在する可能性のあるトレードオフを理解することの重要性について話してきた。次は何をするのだろうか？　次は、どのようなマイクロサービスを作るかについて合理的に考え始められるように、どのような機能の断片をサービスに抽出したいのかを把握する必要がある。既存のモノリシックシステムを分解する際には、何らかの形で論理的な分解を行わなくてはならない。そこでドメイン駆動設計の出番だ。

2.10　ドメイン駆動設計

1章では、サービスの境界を定義する上で重要な概念としてドメイン駆動設計を紹介した。ドメインモデルを開発することは、分解の優先順位をどのようにつけるかを考えるときにも役立つ。図2-4に、Music Corpのための高レベルなドメインモデル例を示す。この図にあるのは、ドメインモデリング作業の結果として特定された境界づけられたコンテキストの集合だ。これらの境界づけら

れたコンテキストには、その間に明確な関連が見られる。これは組織自体の相互作用を表していると考えられる。

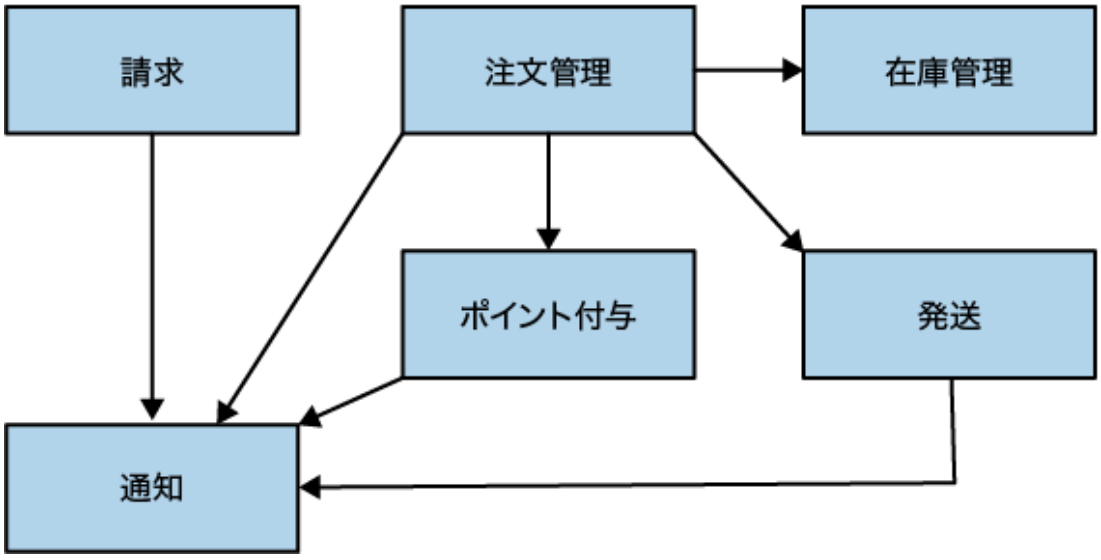

図2-4　Music Corpの境界づけられたコンテキストとそれらの関連

それぞれの境界づけられたコンテキストは、分解できる可能性のある単位を表している。前に説明したように、境界づけられたコンテキストはマイクロサービスの境界を定義するための素晴らしい出発点となる。ということで、既に優先順位を付けるべき対象はリストアップされた。しかし、これらの境界づけられたコンテキスト間の関連もまた有用な情報だ。これは、機能の異なる部分を抽出することの相対的な難しさを評価するのに役立つ。この考え方については後ほど説明する。

マイクロサービスの移行を構造化する上で、ドメインモデルを作成することはほぼ必須のステップだ。しかし、このようなビューを直接作成した経験が無い人は多い。多くの場合、このことが難点となる。そうした人々は、それにどれだけの作業が必要になるかもひどく心配する。たしかにこうした論理的なモデルを作成する上で経験は大いに役立つ。その一方で、たとえ経験が無くても、実際には、ほんの少し労力をかけるだけで、こうしたビューからは本当に有用な恩恵が得られる。

2.10.1　どこまでやればいいか？

既存システムの分解にアプローチするには、根気が必要だ。既存のシステムは、多くの人の手によって構築され、継続的に手を入れられ続けており、より多くの人によって日常的に実際に使用されてもいるだろう。その範囲を考える

と、システム全体の詳細なドメインモデルを作成しようとするのは大変なことかもしれない。

ドメインモデルからは、どこから分解に着手すべきか合理的に判断するのに**必要十分な**情報を得られれば良い、ということを理解するのが重要だ。システム内で最も注意を必要とする部分については、すでにいくつかのアイデアを持っていることだろう。それであれば、必要十分な情報を得るには、モノリスをおおまかな機能のまとまりと捉え、その中からさらに深く掘り下げたい部分を選択すれば十分かもしれない。システムの一部だけを見てしまうと、対処が必要なより大きなシステムの問題を見逃す危険性が生じるが、しかし、それはあまり気にしすぎなくてよい。最初から正確に行う必要はないからだ。必要なのは、次のステップに進むのに十分な情報だけだ。より多くの情報を得るにつれ、ドメインモデルを継続的に洗練させられる。新しい機能が公開されたなら、それを反映させて常にドメインモデルを最新の状態に保とう。

2.10.2　イベントストーミング

Alberto Brandoliniによって考案された**イベントストーミング**は、技術的なステークホルダーと非技術的なステークホルダーが一緒に共有ドメインモデルを定義する共同作業だ。イベントストーミングはボトムアップで行う。参加者はまず、システム内で発生する「ドメインイベント」を定義することから始める。次に、これらのイベントを集約にまとめ、そして集約を境界づけられたコンテキストへとまとめていく。

ここで重要なのは、イベントストーミングは、イベント駆動型のシステムを構築するためのものではないということだ。イベントストーミングは、システムでどのような（論理的な）イベントが発生するかを理解することに焦点を当て、システムのステークホルダーとして関心を持っている事実を特定するものだ。これらのドメインイベントは、イベント駆動型システムの中で発生するイベントに対応させることも可能だが、異なる方法で表現することもできる。

Albertoがこの手法で本当に焦点をあてていることの一つは、モデルを共同で定義するという考え方だ。この作業の成果物はモデルそのものではなく、モデルについての**共通理解**だ。この作業を成功させるには、適切なステークホルダーをその場に集める必要があり、それが最大の課題となることが多い。

イベントストーミングの詳細については本書の範囲外だが、私自身はよく使うテクニックで、とても気に入っている。もっと詳しく知りたい場合には、

Albertoの “Introducing EventStorming”（Leanpub）[5]を読むといいだろう。

2.10.3　ドメインモデルを使った優先順位付け

図2-4に示す図からは、いくつかの有用な示唆が得られる。上流または下流の依存関係の数に基づけば、どの機能が抽出しやすいか、あるいは抽出しにくいかを推測できる。たとえば、通知機能の抽出を考えるとする。この場合、図2-5に示すように、通知機能に対する多くの内向きの依存関係があることが分かる。システムの多くの部分がこの機能を必要としているということだ。つまり、通知サービスを新たに抽出するには、既存のコードで多くの作業をしなければならなくなる。3章では、こうした種類の変更に関する複数のテクニックを見ていく。

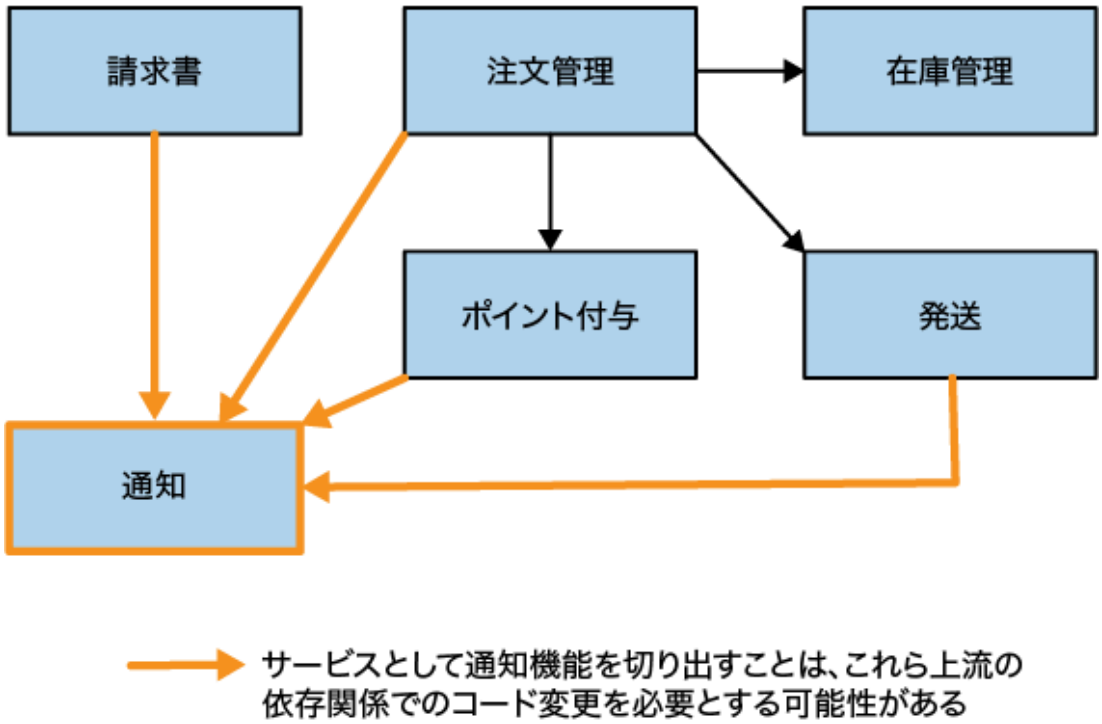

図2-5　通知機能はドメインモデルの観点から論理的に結合されているように見えるため、抽出が困難になる可能性がある

通知機能の抽出はどうやら良いスタート地点ではなさそうだ。一方で、図2-6に示すように、請求書機能はシステムの動作を抽出するのがはるかに容易かもしれない。内向きの依存関係がないため、既存のモノリスに対する変更が少なくて済むからだ。「パターン：ストラングラーアプリケーション」などのパターンは、このような場合に効果的かもしれない。次の章では、ストラングラーパターンを始めとする多くのパターンについて探っていく。

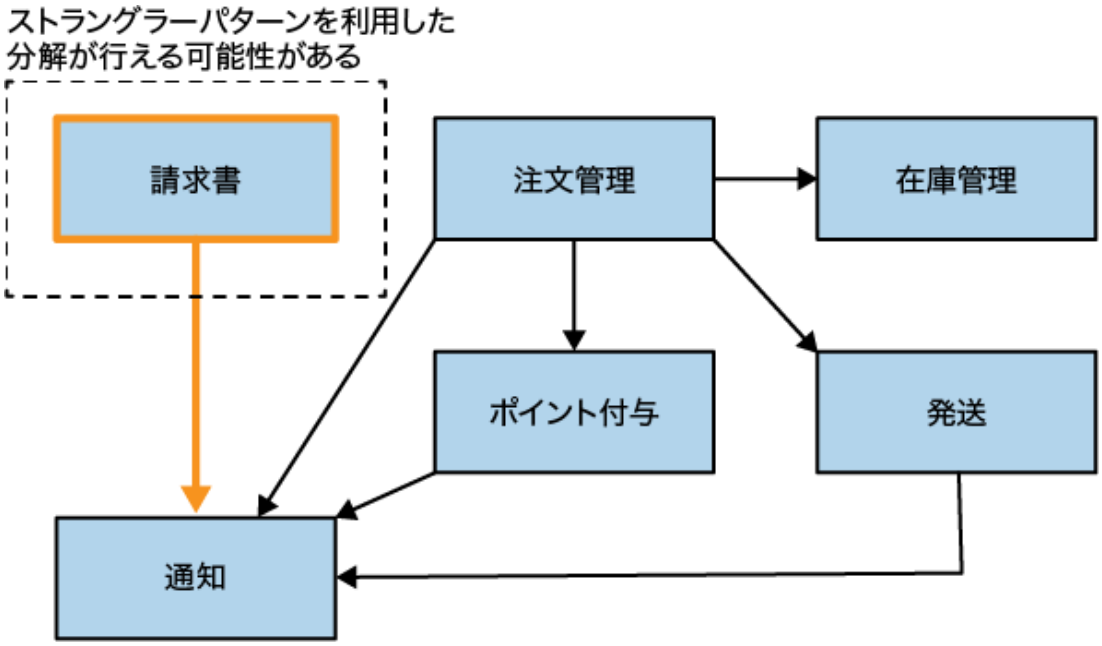

図2-6　請求書機能のほうが抽出しやすい

抽出の難しさを評価する際には、こうした関係性から始めるのが良い方法だが、このドメインモデルは既存のシステムの**論理**ビューであるということを理解しておく必要がある。これは、モノリスの基礎となるコード構造がこのように構造化されていることを保証するものではない。つまり、論理モデルは、より多く（またはより少なく）結合されている可能性が高い機能の断片を導くのには役立つが、機能のもつれ具合の実際をよりよく評価するには、コード自体を見る必要があるかもしれないということだ。このようなドメインモデルだけでは、どの境界づけられたコンテキストがデータベースにデータを保存しているかは分からない。請求書機能が大量のデータを管理していたとすると、データベースの分解作業の影響を考慮する必要がでてくる。4章で説明するように、モノリシックなデータストアを分解することは可能だし、そうすべきだ。しかし、それはマイクロサービスが数個しかない段階から始めたいものではないかもしれない。

何が簡単そうで何が難しそうかというレンズを通して物事を見られるため、こうした考察には価値がある。結局のところ、迅速に成果をあげられるところを見つけたいからだ。しかし、マイクロサービスは特定の何かを達成するための手段であることを忘れてはいけない。たとえば、請求書機能の抽出がマイクロサービス化の簡単な最初の一歩のように見えていたとする。しかし、市場投入時間を短縮することが目標で、請求書機能がほとんど変更されない機能である場合には、請求書機能の抽出は時間の使い方として適していないかもしれない。

したがって、何が簡単で何が難しいかという視点と、マイクロサービス分解がもたらすメリットという視点を組み合わせる必要がある。

2.11　組み合わせモデル

私たちは、素早く進め、勢いに乗り、アプローチの有効性について早期にフィードバックを得るために、いくつか迅速な成果を出したいと考えている。これにより、抽出しやすいものを選びたいと思うようになる。一方で、分解することで何らかの効果を得ることも必要だ。では、これらをどう考慮すればいいのだろうか？

基本的には、どちらの優先順位付けも意味はある。そのため、両方を一緒に可視化して適切なトレードオフを行う仕組みが必要となる。私はこのために、図2-7に示すようなシンプルな構造を使うのが好みだ。抽出される各候補サービスを示されている2つの軸に沿って配置する。x軸は分解によってもたらされると考える価値を表し、y軸は難易度を表している。

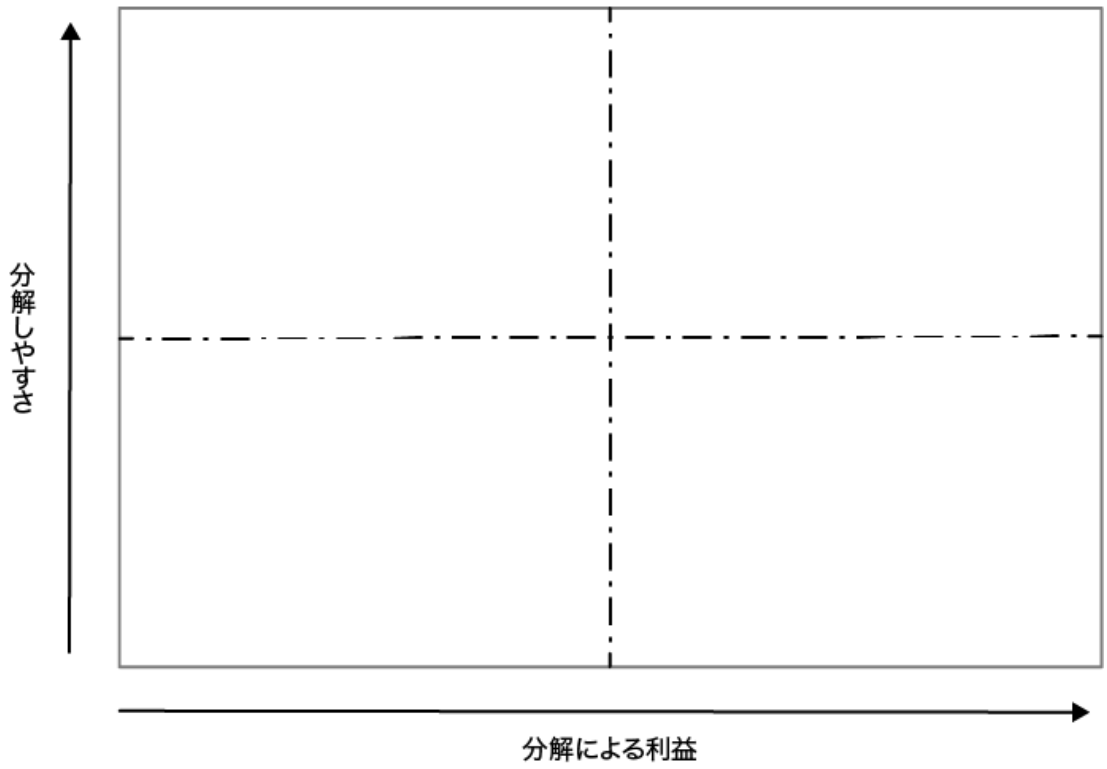

図2-7　サービス分解に優先順位を付けるための単純な2軸モデル

このプロセスをチームで行うことによって、何が抽出に適した候補になるかを見つけ出せる。優れた象限モデルが全てそうであるように、望ましいものは右上に示される。図2-8にそれを示す。請求書機能を含む、その辺りにある機能は、私たちが抽出しやすく、何らかの利益をもたらしてくれると考えている機

能を表している。そこで、最初に抽出するサービスとして、このグループから1つ（あるいは2つ）のサービスを選ぼう。

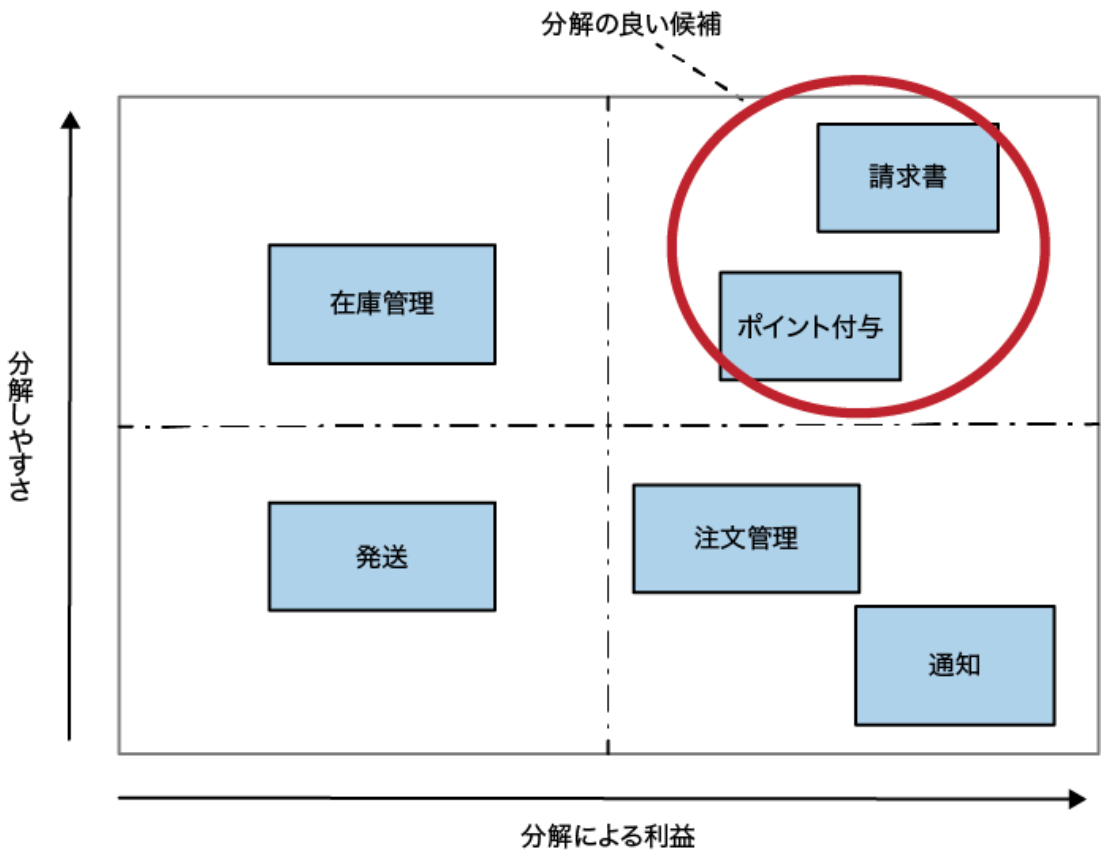

図2-8　優先順位付け象限の利用例

変更を加えていくうちに、より多くのことが分かるようになるだろう。簡単だと思っていたことの中には、難しいこともでてくるかもしれない。逆に、難しいと思っていたことが、簡単になることもある。これは自然なことだ。しかし、それは、より多くのことを学ぶにしたがって、この優先順位付けを見直し、計画を立て直すことが重要だということを意味する。もしかしたら、モノリスを少しずつ削っていくうちに、通知機能が思ったよりも簡単に抽出できることに気付くかもしれない。

2.12　チームを再編成する

以降の章では、マイクロサービスへの移行を成功させるためにアーキテクチャとコードを変更することに主に焦点を当てていく。しかし、既に説明したように、アーキテクチャと組織の整合性を取ることが、マイクロサービスアーキテクチャを最大限に活用するための鍵となる。

とはいえ、組織を変えなければ、そうした新しいアイデアを活用できない状況に陥っている可能性もある。組織の変革についての詳細な研究は本書の範囲外

だが、より深い技術的側面へと飛び込む前に、いくつかのアイデアを残しておくことにする。

2.12.1　構造を変える

歴史的には、IT組織はコアコンピテンシーを中心に構成されていた。Java開発者は他のJava開発者とチームを組み、テスターは他のテスターとチームを組み、そしてDBAは全員で1つのチームに属していた。ソフトウェアを作成する際には、そうしたチームから短期的な取り組みに従事すべくメンバーが割り当てられていた。

そのため、ソフトウェアを作成するという行為は、チーム間でバトンを受け渡していくことを必要としていた。ビジネスアナリストは顧客と話をして、顧客が何を望んでいるかを見つける。アナリストはその後、要件を書き上げ、開発チームに渡す。そして、開発チームは作業を行い、いくつかの作業を終えたら、ソフトウェアをテストチームへと渡す。テストチームが問題を発見した場合は、それを送り返す。問題がなければ、ソフトウェアは運用チームに送られ、デプロイされるといった具合だ。

このようなサイロ化はよく知られているものだ。前の章で説明した階層化アーキテクチャを考えてみよう。階層化されたアーキテクチャでは、単純な変更を公開する際に複数のサービスの変更が必要になることがある。組織的なサイロも同様だ。ソフトウェアの作成や変更に関与しなければならないチームが多ければ多いほど、時間がかかることになる。

しかし、このようなサイロは崩壊しつつある。多くの組織では、専用のテストチームはもはや過去のものとなっている。その代わりに、テストの専門家はデリバリーチームに統合され、開発者とテスターがより密接に連携して作業できるようになってきている。また、DevOpsのムーブメントによって、多くの組織は中央集権的な運用チームから離れ、代わりに、運用上の考慮事項についてより多くの責任をデリバリーチームが受け持つようになった。

専門チームの役割がデリバリーチームに押し込まれるようになり、中央集権的なチームの役割も変わってきている。中央集権的なチームは自分たちで作業を行うのではなく、代わりにデリバリーチームが作業を行うのを支援するようになったのだ。これには、専門家をチームに組み込んだり、セルフサービスツールを作成したり、トレーニングを提供したりといった多くの活動が含まれる。

中央集権的なチームの責任は、「やること」から「可能にすること」へと変化していっている。

その結果、独立した自律的なチームがますます増えてきており、これまで以上にデリバリーサイクル全体に責任を持てるようになってきている。デリバリーチームの焦点は、特定の技術や活動ではなく、製品のさまざまな領域へと広がっている。これは、技術志向のサービスから、ビジネス機能の垂直スライスに基づいてモデル化されたサービスへと切り替わっていっているのと同様だ。ここで理解すべき重要なことは、この移り変わりは何年も前から明らかになっている明確な傾向だが、絶対的なものではなく、急速な変革というわけでもないということだ。

2.12.2　どこにでもフィットするやり方はない

この章は、マイクロサービスを使用するかどうかの決定が、直面している課題と、それを実現したいと考えている変化にどう根ざしているかを議論することから始まった。組織構造を変えることも同様に重要だ。組織を変える必要があるかどうか、そしてどう変える必要があるかを理解するには、組織が置かれている文脈、労働文化、そしてそこにいる人々に基づいて考える必要がある。これが、他所の組織デザインをただ真似ることが特に危険な理由だ。

Spotifyモデルについては少し前に軽く触れた。Spotifyがどのように組織を編成しているかに関心が高まったのは、Henrik KnibergとAnders Ivarssonによる2012年の有名な論文 “Scaling Agile @ Spotify”（http://bit.ly/2ogAz3d）[21]からだ。この論文は、スクワッドやチャプター、ギルドといった概念を広めた論文だ。これらの用語は、誤解されているとはいえ、今では私たちの業界では当たり前になっている。Spotifyが内部で使用したことのない用語であったにもかかわらず、最終的にはこのモデルは「Spotifyモデル」と呼ばれるようになった。

その後、この構造を採用する企業が急増した。しかし、マイクロサービスと同様、Spotifyがどのような文脈で運営されているのか、自社のビジネスモデルや課題、企業文化などを十分に考慮せずに、多くの組織がSpotifyモデルに傾倒してしまったのだ。スウェーデンの音楽ストリーミング企業でうまくいっていた組織構造であっても、投資銀行で機能しない可能性は十分にある。さらに、元の論文は、2012年にSpotifyがどのように機能していたかのスナップショットが示されていたものだった。その後状況は変化し、結局はSpotifyでさえもSpotifyモデルを使わなくなったことが明らかになっている。

同じことを、あなたにも当てはめる必要がある。他の組織が行ってきたことからインスピレーションを得ることは確かにある。しかし、他の組織が行ってきたことが自分の文脈でも通用すると決めつけてはいけない。Jessica Kerrが、かつてSpotifyモデルに関連して言及したとおりだ。「答えではなく、問いを模倣しよう」[†10]。Spotifyの組織構造のスナップショットは、Spotifyが自分たちの問題を解決するために実施した変更を反映したものだ。あなたのやり方で、柔軟に疑う姿勢を模倣し、新しいことを試してみよう。ただし、あなたが行う変更は、あなたの会社やそのニーズ、会社にいる人々や文化の理解に根ざしている必要がある。

[†10] http://bit.ly/2AKTaXP

具体例を挙げよう。多くの企業がデリバリーチームに対して、「ソフトウェアをデプロイし、24時間365日サポートを行っていくことは今や必須だ」と言っているのを目にする。こうした発言は、信じられないほど破壊的で、役に立たないこともある。ときに大胆で思い切った発言をすることは、物事を動かすための素晴らしい方法であることも分かる。しかし、それがもたらす混乱に備えよう。もし開発者が電話がならない環境で9時から17時まで働くことに慣れていて、サポートや運用を行ったことがなく、SSHのことをさっぱり分かっていないようなら、こうした発言はスタッフを疎外し、多くの人を失う最悪なやり方といえる。もしそれが組織にとって正しい変更だと考えるのなら、問題ない。それを野望、達成したい目標としてみんなに話し、その理由を説明しよう。その上で、みんなで一丸となってその目標に向かって旅をするようにしよう。

チームがソフトウェアのライフサイクル全体をより完全に自分のものにしたいと本気で考えているのであれば、それを行えるようにチームのスキルを変える必要があることを理解しなくてはならない。ヘルプやトレーニングを提供したり、新しい人をチームに加えたりできるはずだ（おそらく、運用チームの誰かをデリバリーチームに組み込むこともできるだろう）。ソフトウェアと同じように、どのような変化をもたらしたいかに関わらず、それは段階的に実現することが可能だ。

DevOpsはNoOpsという意味ではない

DevOpsという用語をめぐっては、混乱が広がっている。それを、開発者が全てのオペレーションを行うことを意味し、運用は不要だと思い込んでいる人もいる。しかし、これは事実とはかけ離れた理解だ。基本的に、DevOpsとは開発と運用の間の垣根を取り払うというコンセプトに基づいた文化的運動だ。これらの役割のスペシャリストが必要な場合もあれば、そうでない場合もある。しかし、何をするにしても、ソフトウェアの提供に関わる人々（それぞれの責任が何であれ）を横断した共通の連携と理解を促進したいというのが、DevOpsの考えだ。

これについての詳細は “Team Topologies”（IT Revolution Press）[30]をお勧めする。この書籍はDevOpsの組織構造を探った書籍となっている。範囲は広くなるものの、このトピックに関するもう1つの優れたリソースは『The DevOpsハンドブック』（日経BP）[19]だ。

2.12.3　変化を起こす

他所の構造を真似してはいけないのだとしたら、どこから始めればいいのだろうか？　デリバリーチームの役割を変えようとしている組織と仕事をするとき、私はまず、その企業内でソフトウェアのデリバリーに関わる全ての活動と責任を明確にリストアップすることから始めるようにしている。そして、その次に、それらの活動を既存の組織構造にマッピングする。

もし既に本番環境への道筋を、私が好むようなやり方でモデル化しているのであれば、既存のビューに所有権の境界線をオーバーレイできる。あるいは、図2-9のようなシンプルなものでも良いだろう。関係する全ての役割からステークホルダーを集め、ソフトウェアデリバリーに関わる全ての活動を一つのグループとして、社内でブレインストーミングしよう。

図2-9　デリバリー関連の責任のサブセットと、それらが既存のチームにどのように対応しているか

現在の「AS-IS」の状態を理解しておくことは、関係する全ての作業を全員で共有するのに役立つため、とても重要だ。サイロ化された組織の性質として、あるサイロが何をしていて、別のサイロが何をしているのかを理解するのに苦労することがある。そのため、「AS-IS」の理解は、どれくらい早く変化できるかについて、組織が正直になるのに役立つと私は感じている。それによって、全てのチームが同じではないことに気付けるだろう。あるチームは既に多くのことを自分たちでやっているかもしれないし、別のチームはテストからデプロイまで全てを他のチームに頼っているかもしれない。

デリバリーチームがテストやユーザーテストのためにソフトウェアを既にデプロイしている場合、本番環境へのステップはそれほど大きくないかもしれない。一方で、ポケベルを持ち歩いて一次対応することや本番環境の問題の診断などを引き受けることの影響を考慮しなければならない。これらのスキルは人々の長年の仕事によって構築されたものであり、開発者が一晩で対応できるようになることを期待するのは全く現実的ではない。

現状を把握したら、将来的にどうあるべきかというビジョンを持って、合理的なタイムスケールの範囲内で物事を描き直そう。詳細を探りたいのなら、その範囲は半年から1年では長すぎるかもしれない。どの責務の所在が変わっているだろうか？　どのように移行を実現するのだろうか？　移行に必要なものは何か？　チームにはどんな新しいスキルが必要だろうか？　さまざまな変化に対する優先順位はどうなるだろうか？

先ほどの例では、図2-10に示すように、フロントエンドチームとバックエンドチームの責任を統合することにした。また、チームは独自のテスト環境を提供できるようにしたいと考えている。しかし、そのためには、運用チームがデリバリーチームにセルフサービスのプラットフォームを提供する必要がある。私たちは、最終的にソフトウェアの全てのサポートをデリバリーチームに任せたいと考えている。自分たちのテストデプロイを自分たちのものにすることは、良い第一歩だ。また、全てのインシデントは営業日中に処理することにし、既存の運用チームがコーチングを行う安全な環境で、そのプロセスに慣れるチャンスを与えることにした。

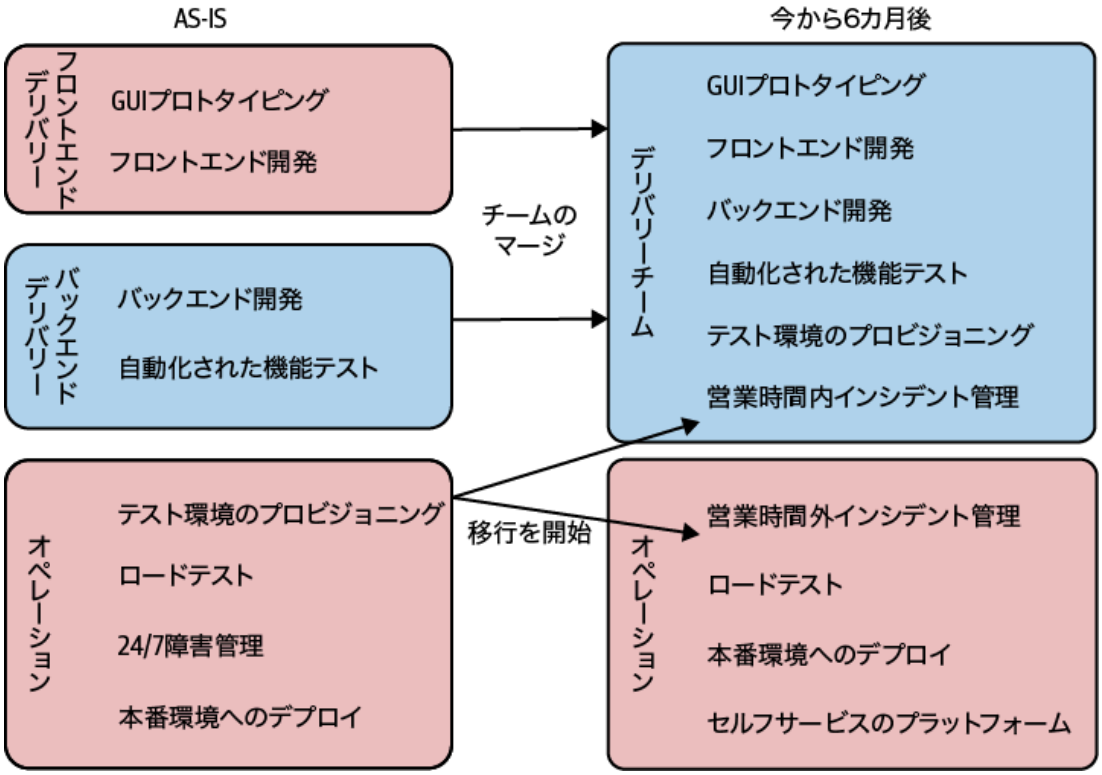

図2-10　組織内で責任を再割り当てする例

全体像を見ることは、もたらしたい変化のきっかけにとても役立つが、その変化が実現可能かどうか、実現可能であればどのようにして実現するのか、現場の人々と一緒に時間をかけて検討する必要がある。特定の責任の間で物事を分割することで、この変化に対して段階的なアプローチを取ることもできる。運

用チームがテスト環境を提供する必要性をなくすことにまず注力することが、あなたにとって正しい第一歩だ。

2.12.4　スキルを変える

必要なスキルを評価してギャップを埋める支援を行う際、私はよく自己評価をしてもらうことにしている。それを使い、チームがその変化を実行するためにどのようなサポートが必要なのかをより広く理解するやり方が好きだからだ。

具体例として、ThoughtWorksに在籍していた時に関わったプロジェクトのことを話そう。私たちは、ガーディアン紙のオンラインプレゼンスの再構築を支援するために雇われていた（これについては次の章で説明する）。そのプロジェクトでは、新しいプログラミング言語や関連技術を用いて開発スピードを上げる必要があった。

プロジェクトの開始時に、私たちのチームは、ガーディアン紙の開発者が取り組むべき重要なコアスキルのリストを作成した。それぞれの開発者は、これらの基準に照らし合わせて自分自身を評価し、1（「私には何の意味もない！」）から5（「これについての本を書ける」）までのランク付けを行った。各開発者のスコアは非公開で、メンターだけが共有できるようにした。この作業は、各開発者がそれぞれのスキルを5にすることではなく、開発者自身が達成したい目標を自分たちで設定できるようにすることを目的としていた。

コーチとしての私の仕事は、コーチングしている開発者の1人がオラクルのスキルを向上させたいと考えている場合に、その開発者にその機会を提供することだった。これには、その技術を利用したストーリーに取り組んでいるかどうかを確認したり、ビデオを推薦して見てもらったり、トレーニングコースやカンファレンスへの参加を検討したりすることなどが含まれていた。

このプロセスを通して、個人が時間と労力を集中させたい分野を視覚的に表現できる。図2-11に例を示そう。これは、私がKubernetesとLambdaの経験を増やすことに時間とエネルギーを集中させたいということを表していて、それはおそらく、ソフトウェアのデプロイを自分自身で管理しなければならない状況にあるという事実を示している。また、このプロセスで同時に重要なのは、自分が満足している分野を強調することだ。この例では、Goのコーディングは今のところ焦点を当てる必要がないと私が感じていることを表している。

この種の自己評価を非公開にしておくことはとても重要だ。ポイントは、誰かが他の誰かを評価することではなく、人々の自己開発を導く手助けをすることにあるからだ。これを公にしてしまうと、この作業のパラメーターが大幅に変更されることになる。たとえば、業績評価に影響を与えることを懸念し、自分自身に低いスコアを与えることを人々は心配するだろう。

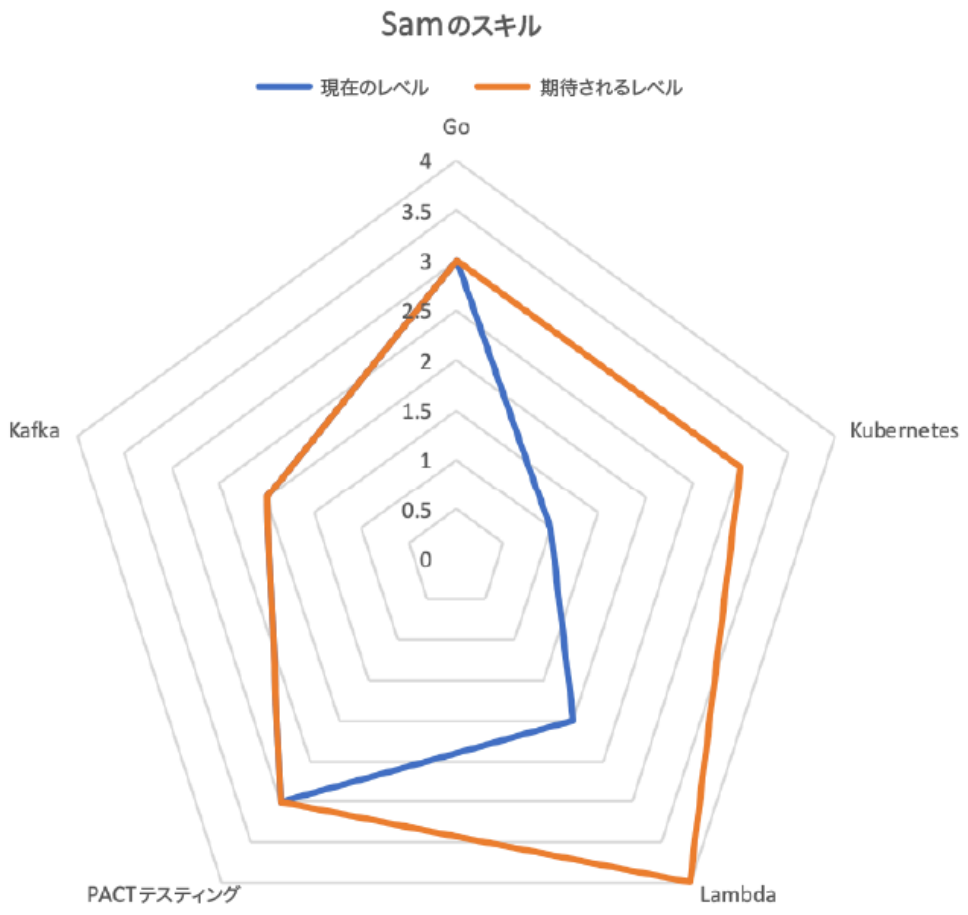

図2-11　スキルチャートの例：私（著者）が改善したい分野を示すスキルチャートの例

各人のスコアは非公開としたとしても、これを使用してチーム全体のイメージを構築することはできる。匿名化された自己評価の結果をもとに、チーム全体のスキルマップを作成する。これは、より体系的なレベルで取り組む必要のあるギャップを浮き彫りにするのに役立つ。図2-12では、自分のPactテスティングのスキルレベルには満足しているかもしれないが、チーム全体としてはその分野でもっと向上したいと考えていることが分かる。これにより、グループ学習の必要性が浮き彫りになり、内部トレーニングコースを実行するなどの大規模な投資が正当化されるかもしれない。また、この全体像をチームと共有することは、チーム全体のバランスを保つためにそれぞれのメンバーがどのように貢献できるかを理解する助けにもなるだろう。

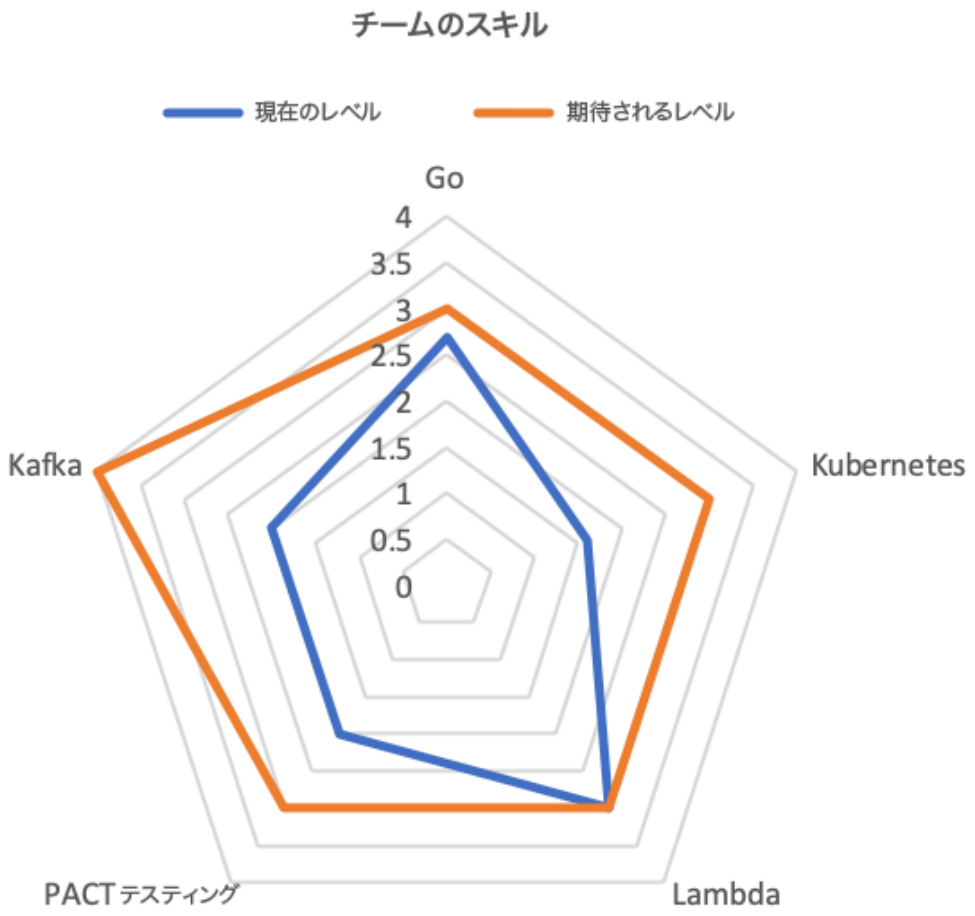

図2-12　全体として見ると、チームはKafka、Kubernetes、Pactテスティングのスキルを向上させる必要がある

もちろん、既存のチームメンバーのスキルセットを変えることだけが前進の道ではない。多くの場合、私たちが目指すのは、チーム全体がより多くの責任を負うデリバリーチームだ。しかし、それは個々人がより多くのことをしているというわけではない。必要なスキルを持った新しい人をチームに迎えることが正しい答えかもしれない。チームメンバーがKafkaについて学ぶのを手伝うのではなく、Kafkaの専門家を雇ってチームに参加させることもできる。そうすれば、短期的な問題を解決でき、チーム内の専門家がこの分野で同僚の学習を支援することもできるだろう。

このトピックをもっと探っていくこともできるが、ここまでであなたが始めるのに十分な情報を共有できたと願っている。組織の変革は、組織の人々や文化、ユーザーのニーズを理解することから始まる。他所の事例に触発されるのは構わないが、他所の問題に対する答えを真似してうまくいかなかったからといって驚いてはいけない。

2.13　うまくいっているかどうかを分かるには？

私たちは間違いを犯すものだ。たとえ最善だと思ってマイクロサービスへの旅を始めたとしても、全てを分かっているわけではなく、うまくいかないと気付く可能性も受け入れなければならない。ここで疑問に思うことがある。うまくいっているかどうかはどうやったら分かるだろうか？　そして、私たちは間違いを犯してきていないだろうか？

達成したい成果に基づいて、これらの質問に答えるのに役立つ追跡可能な指標を定義してみよう。指標例のいくつかはこの後すぐに探っていくが、追跡可能な指標が定量的な指標だけではないということを、ここで強調しておきたい。うまくいっているかどうかを知るには、現場の人々からの定性的なフィードバックも考慮しなくてはならない。

これらの定量的、定性的な指標は、継続的なレビュープロセスに通す必要がある。チェックポイントを設定して、チームが正しい方向に向かっているかどうかをふりかえる時間を確保する必要がある。このチェックポイントで自問するのは、「これはうまくいっているか？」ではなく、「代わりに何か他のことを試してみるべきだろうか？」ということだ。

ここでは、チェックポイントでの活動をどのように構成するかと、追跡できる指標の例を見ていこう。

2.13.1　定期的なチェックポイントを設ける

移行では、デリバリープロセスの中に、立ち止まってふりかえるための時間を組み込むことが重要だ。そこでは利用可能な情報を分析し、軌道修正が必要かどうかを判断する。小規模なチームの場合、これは非公式なものであってもよいし、定期的なふりかえりに組み込んでもよい。大規模なプログラムであれば、明確な定期の活動として計画するか、さまざまな活動のリーダーを集めての月に一度のセッションで状況をふりかえる必要があるかもしれない。

こうした活動の実施の頻度にかかわらず、また、どれだけ形式的（または非公式）に行うにしても、私は次のことを確実に行うことをお勧めしている。

1. マイクロサービスへの移行で何を達成したいと期待しているのかを再確認する。ビジネスの方向性が変わってしまって、あなたの進む方向性が意味をなさなくなってしまったのであれば、その時点でやめることだ。

2. 導入している定量的な手段を確認して、進歩しているかどうかを確認する。
3. 定性的なフィードバックを求める。みんなは物事がまだうまくいっていると思っているだろうか？
4. もし何かあるなら、今後変えていくことを決める。

2.13.2　定量的な測定

進捗を追跡するために選択する指標は、達成しようとしている目標によって異なる。たとえば、市場投入時間を改善することに焦点を当てているのであれば、サイクルタイム、デプロイ数、障害率を測定することは理にかなっている。より多くの負荷を処理するためにアプリケーションをスケーリングしようとしているのであれば、最新のパフォーマンステストの報告を返すことは理にかなっているだろう。

メトリクスには危険性があるということには注意したい。ことわざにもあるとおり、「計測したものが得られる」からだ。メトリクスは、意図せずに、または意図的にゲーム化する可能性がある。このことに関しては、妻が働いていた会社の話を思い出す。その会社は、外部ベンダーが閉じたチケット数を追跡しており、その結果に基づいて支払いを行っていた。そこで何が起きたかというと、ベンダーは問題が解決しなかった場合でもチケットを閉じ、代わりに新しいチケットを開いていたのだ。

メトリクスは短期間では変化しにくいかもしれない。マイクロサービス移行の最初の数カ月でサイクルタイムが大幅に改善されたとしたら、私はきっと驚くことだろう。物事の進め方を変えると、チームが新しい作業方法に慣れるまでの間、短期的には生産性にマイナスの影響を与えることがよくあるからだ。これが、少しずつ段階を踏んでいくことが重要なもう一つの理由だ。変化が小さければ小さいほど、可能性のあるネガティブな影響が小さくなり、それが発生したとしても素早く対処できるようになる。

2.13.3　定性的な測定

> *……ソフトウェアは感情でできている。*
>
> *Astrid Atkinson（@shinynew_oz）*

データが示すものは何であれ、ソフトウェアを構築するのは人であり、そうした人々のフィードバックが成功の測定に含まれていることが重要だ。彼ら彼女らはプロセスを楽しんでいるだろうか？　力を与えられたと感じているか？　それとも圧倒されていると感じている？　新しい責任を負ったり、新しいスキルを習得したりするために必要なサポートを受けているだろうか？

この種の移行のためのスコアカードを上層部に報告する[†11]際には、チームがどう感じているかの定性的なチェックを含める必要がある。チームがそれを気に入ってくれているのであればばっちりだ！　もしそうでなければ、あなたはそれについて何かをする必要があるかもしれない。定量的なメトリクスにのみ依存して、人々の意見を無視するのは、あなた自身を多くのトラブルに巻き込むのにうってつけの方法だ。

[†11] そう、これは実際に起きたことだ。楽しいことやKubernetesが全てではないんだ……。

2.13.4　サンクコストの錯誤を回避する

サンクコスト（埋没費用）の錯誤を認識する必要がある。レビュープロセスを持つことは、誠実さを保ち、うまくいけばこの現象を回避するための一部となる。**サンクコストの錯誤**は、何かをするために先行して行っているアプローチにあまりにも投資しすぎた場合に発生する。そうすると、たとえそのアプローチが機能していないことを証拠が示していても、人々はその方法を続ける。そうして、私たちは時に「もうちょっとしたら変わるはずだ！」と自分自身を正当化する。変更を行うために組織内であまりにも多くの政治的資本を注ぎ込んだため、後戻りできなくなってしまうということもあるだろう。いずれにせよ、それまでの考えにとらわれているために諦めきれないという、情緒的な投資がサンクコストの錯誤へと繋がるのは論をまたない。

私の経験では、賭けが大きければ大きいほど、そしてそれに伴うファンファーレが大きければ大きいほど、それがうまくいかなくなったときに手を引くのが難しくなる。サンクコストの錯誤はまた、コンコルドの誤りという名前でも知られている。それは、イギリスとフランスの政府が多額の費用で超音速旅客機を構築するために多大な費用をかけて失敗したプロジェクトに由来する。このプロジェクトが決して金銭的なリターンをもたらすことはないという全ての証

拠にもかかわらず、プロジェクトにはますます多くのお金が投入された。工学的な成功はともかくとして、コンコルドは商業飛行のための実用的な航空機としては決して成功しなかった。

一つ一つのステップを小さなものにしていけば、サンクコストの錯誤による落とし穴を回避しやすくなる。そうすることで、軌道修正が容易になるからだ。前に議論したチェックポイントの仕組みを使って、何が起こっているかをふりかえろう。トラブルの最初の兆候で手を引いたり、軌道修正したりする必要はないが、あなたがもたらそうとしている変化の成功（またはその他）に関して収集している証拠を無視することは、証拠を収集しないよりも間違いなく愚かな行いだ。

2.13.5　新しいやり方への選択肢を開いておく

ここまで読んでくれていれば驚かないと思うが、モノリシックなシステムや私たちが取りうる経路を分解するには、いくつかの変数が関係する。ただ一つ確かなことは、全てがスムーズに行くわけではないということだ。そのため、行った変更を取りやめて新しいことを試すのにオープンである必要があるし、ときには、行った変更がもたらす影響を確認するために、しばらく我慢して様子を見る必要もあるだろう。

あなたが常に新しい何かに挑戦するために、絶え間ない改善の文化を受け入れようとするなら、必要に応じて方向性を変えることはずっと自然なことになる。もし、変化やプロセス改善の考えを、普段行っていること全ての中に組み込むのではなく、独立した作業の流れに追いやってしまうと、変化のための活動を一度限りのものとみなしてしまう危険性がある。そうなると、その作業が完了したら、私たちにとってこれ以上の変化はない！というように、それっきりとなってしまう。そのような考え方では、数年後には競合他社に後れをとり、また別の山に登ることになるだろう。

2.14　この章のまとめ

この章の内容は多岐にわたっていた。まず、マイクロサービスアーキテクチャを採用したいと思う理由と、その意思決定が時間の使い方を決定する優先順位付けにどのような影響を与えるかを見てきた。そして、マイクロサービスが適切な選択かを判断する際に、チームが必ず自問しなくてはならない重要な質問事項を検討した。

- 達成したいことは何か？
- マイクロサービスの他に代替案はなかったか？
- どうすれば移行がうまくいってるか分かるだろうか？

さらに、マイクロサービスを抽出する際に段階的なアプローチを採用することの重要性は、強調してしすぎることはない。間違いは避けられないので、それを当たり前のように受け入れ、大きな間違いではなく小さな間違いをすることを目指すべきだ。マイクロサービスアーキテクチャへの移行を小さく段階的なステップに分割することで、間違いは確実に小さくなり、取り返しがつきやすくなる。

私たちの多くは、実際の顧客がいるシステムでも仕事をしている。顧客が使用している既存のアプリケーションを放置したまま、アプリケーションのビッグバンリライトに何カ月も何年も費やす余裕はない。目標は、新しいマイクロサービスを少しずつ作成し、本番環境のソリューションの一部としてデプロイすることで、経験から学び、できるだけ早く利益が得られるようにすることだ。

機能を新しいサービスへと抽出する作業は、本番環境で使われ始めるまで完了とは言えない。最初のいくつかのサービスを実際に**使ってみる**というプロセスからは、多くのことを学べる。初期の段階では、そこに焦点を当てる必要がある。

つまり、新しいマイクロサービスを作成し、（できれば）縮小しつつあるモノリスと統合し、本番環境に反映させるための一連のテクニックを開発する必要があるということだ。次の章では、システムを常に稼働させ、顧客にサービスを提供しつつ、新しい機能も導入しながら、モノリスからマイクロサービスに移行していくためのパターンを見ていく。